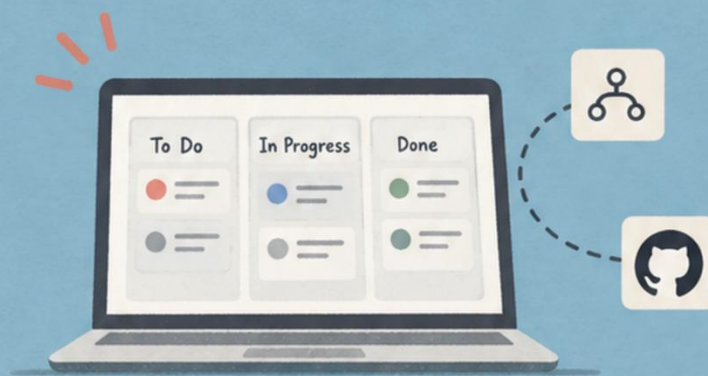




SOFTWARE PROJECT MANAGEMENT (SPM)

Personal Notes and References

Anjan Khadka



Before Getting Started

For smooth flow and understanding:

Heading 1 (units and main heading)

Heading 2 (Topics and headings)

Heading 3 (subheadings)

Important

Supporting

Examples, idea.

Laughing Through the Chaos (Section):



A short, humorous summary that turns dry concepts into memorable ideas using real examples and simple stories — so revision hurts less and the important points stick longer.



Table of Contents

Before Getting Started	2
Table of Contents	3
UNIT 1: Software Project Management Concepts	6
1.1 Introduction to Projects & Software Projects	6
1.2 Software Projects vs. Other Engineering Projects	8
1.3 Importance and Problems in Software Project Management	9
1.4 SPM Process & Project Lifecycle.....	13
1.5 Characteristics of a Good & Successful Software Project Manager	14
Laughing Through the Chaos - Unit 1 Reality Check.....	19
UNIT 2: Software Project Scheduling.....	21
2.1 Objectives of Activity Planning	21
2.2 Work Breakdown Structure (WBS).....	22
DEFINITION: WBS Rules & Work Packages	23
2.3 Network Planning Models & Dependencies.....	24
2.3.1 Network Diagram	24
2.5 Scheduling Methods.....	36
2.5.1 Program Evaluation and Review Technique (PERT).....	36
2.5.2 Critical Path Method (CPM) & Float Calculations.....	37
2.6 Shortening Project Duration (Schedule Compression)	38
Laughing Through the Chaos - Unit 2 Reality Check	39
UNIT 3: Software Estimation Techniques	41

3.1 Problems with Over and Under Estimation.....	41
3.2 Software Effort Estimation Techniques	41
KEY FORMULA & METHOD: Function Point Calculation Steps.....	44
Laughing Through the Chaos - Unit 3 Reality Check	47
UNIT 4: Software Evaluation and Costing	49
4.1 Project Evaluation: Strategic & Technical Assessment	49
4.1 Project Evaluation	49
4.1.1 Strategic Assessment	49
4.1.2 Technical Assessment	50
4.2 Cost-Benefit Analysis	50
4.3 Cash Flow Forecasting	51
4.4 Cost-Benefit Evaluation Techniques.....	52
4.5 Risk Evaluation.....	55
4.6 Selection of Appropriate Project Approach	55
4.6.2 Choosing Technologies.....	56
4.6.2 Choice of Process Models.....	56
4.6.3 Structured Methods.....	58
Laughing Through the Chaos - Unit 4 Reality Check	59
UNIT 5: Risk Management [5 Hrs.].....	62
5.1 Risk Concepts & Categories.....	62
5.2 Categories of Risk	62
5. Risk Management Framework (5 Steps).....	64
5.3 Risk Response Strategies for Threats.....	68
Laughing Through the Chaos - Unit 5 Reality Check	69

UNIT 6: Software Quality Management [5 Hrs.].....	70
6.1 Software Quality, TQM & Six Sigma.....	70
6.1.1 Total Quality Management (TQM)	70
6.1.2 Six Sigma	71
6.2 ISO 9126 Software Quality Model	71
6.3 Place of Software Quality in Software Planning.....	74
Laughing Through the Chaos - Unit 6 Reality Check	77
UNIT 7: Software Configuration Management	79
7.1 Concept, Need and Elements of SCM	79
DEFINITION: 4 Elements of SCM & Baselines.....	80
7.2 Version Control Tools Case Study: Git vs. CVS vs. SVN.....	85
Laughing Through the Chaos - Unit 7 Reality Check	87
Git Learning Reference:	89

UNIT 1: Software Project Management Concepts

1.1 Introduction to Projects & Software Projects

A Project is defined by the Project Management Institute (PMI) as a *temporary endeavor undertaken to create a unique product, service, or result*. Unlike routine operational work, a project has a well-defined beginning and end, specific financial and resource constraints, and aims to produce a bespoke deliverable.

- **Project:** A time-bound, unique, resource-constrained, progressive effort aimed at achieving specific target objectives.
- **Software Project:** A specialized sub-category of projects centered around software synthesis, engineering design, coding, testing, and deployment. Software projects possess unique traits such as high flexibility, logical complexity, and total invisibility of intermediate code assets.

Core Characteristics of a Project:

- **Temporary Nature:** Every project has a definite start date and an explicit closure date.
- **Unique Deliverable:** Delivers a product or service that differs in key aspects from existing solutions.
- **Resource Constraints:** Operates strictly within predefined limits of time, cost, manpower, and technology.
- **Progressive Elaboration:** Project details are uncovered and refined continuously in iterative steps as understanding deepens.
- **Inherent Risk:** Due to novelty and constraints, every project contains uncertainties (threats and opportunities).

Core Characteristics of a Software Project:

Software projects have certain characteristics that make them different from many traditional engineering projects. These characteristics influence how software projects are planned, estimated, scheduled, developed, and controlled.

- **Intangibility:** Software is not a physical product that can be easily seen or measured during development. Much of the work exists as requirements, designs, source code, databases, and other logical components.
- **High Complexity:** Software systems can contain many interconnected modules, functions, interfaces, and dependencies. A small change in one part of the system may sometimes affect several other parts.
- **Changing Requirements:** Software requirements may change during development because customers understand their needs better after seeing the system, or because business and technology conditions change. These changes can affect the project's scope, cost, and schedule.
- **High Flexibility:** Software can usually be modified more easily than physical products. However, this flexibility may encourage frequent changes, which can increase development effort and project complexity.
- **Difficult Progress Measurement:** Software development does not always produce visible physical progress. Therefore, progress is usually measured through completed requirements, working features, milestones, tested modules, deliverables, and defect status.
- **Human-Intensive:** Software development depends heavily on skilled people such as analysts, designers, developers, testers, and project managers. Their knowledge, experience, communication, motivation, and teamwork can significantly influence project success.

1.2 Software Projects vs. Other Engineering Projects

Software projects share many project-management principles with civil, mechanical, electrical, and other engineering projects. All require planning, resources, budgets, schedules, quality control, risk management, and coordination. However, software has several characteristics that make software project management particularly challenging.

Four important characteristics often associated with software engineering are **invisibility, conformity, complexity, and flexibility**.

Characteristic	Software Engineering Projects	Traditional Engineering Projects
1. Invisibility	Software cannot be visually inspected. Progress is abstract and logical until runnable components are built.	Physical progress is directly visible (e.g., building foundations, bridge arches).
2. Conformity	Software must conform to existing complex hardware, legacy systems, and changing human requirements.	Physical laws (gravity, thermodynamics) govern systems, forcing standard interfaces.
3. Complexity	Software exhibits high logical complexity per unit cost. No two software parts are identical.	Repeated standardized physical components (e.g., standard steel beams or bricks).
4. Flexibility	Software is infinitely malleable and easy to modify, inviting constant requirement changes.	Physical materials are extremely costly and difficult to alter once constructed.

Why Software Projects Are Difficult to Manage?

The characteristics above interact with one another. Because software is invisible, progress can be difficult to assess. Because it is complex, changes can have unexpected consequences. Because it is flexible, stakeholders may request frequent changes.

Because it must conform to existing systems and requirements, the project may face many external constraints.

Therefore, software project management requires more than simply creating a schedule. The project manager must continuously coordinate people, requirements, technology, cost, time, quality, risks, and changes.

1.3 Importance and Problems in Software Project Management

Why SPM is Critical: Sound SPM practices ensure that software systems are delivered on time, within allocated budget, and meeting required quality standards. Historical industry audits (e.g., Standish Group Chaos Reports) revealed that classical unmanaged software projects suffered from massive cost overruns (frequently >100%), severe schedule delays, or total cancellation.

Importance of Software Project Management

Effective SPM is important because software projects involve uncertainty, complex technical work, limited resources, changing requirements, and dependence on people.

- Meeting project objectives

Project management helps establish clear objectives and ensures that development activities remain aligned with those objectives.

- Managing time and schedule

A project may have a fixed deadline. Scheduling helps determine what activities need to be performed, their dependencies, their durations, and the expected completion date.

- Controlling cost

Software projects have limited budgets. Estimation and cost management help the organization determine how much the project is expected to cost and monitor whether actual expenditure is within acceptable limits.

- **Managing resources**

Software projects require developers, testers, analysts, designers, project managers, infrastructure, software tools, and other resources. SPM helps allocate these resources appropriately.

- **Managing risks**

A project may encounter technical failures, requirement changes, staff turnover, security problems, vendor problems, or schedule delays. Risk management allows potential problems to be identified and addressed before they seriously affect the project.

- **Maintaining quality**

Project management incorporates quality planning, quality assurance, reviews, testing, standards, and monitoring so that the final software satisfies the required expectations.

- **Managing requirements and scope**

Uncontrolled changes can increase project cost and duration. SPM provides mechanisms for understanding, documenting, evaluating, and controlling changes to project scope.

- **Improving communication**

Software projects involve customers, managers, developers, testers, business analysts, vendors, and other stakeholders. Effective communication ensures that information reaches the right people at the right time.

- **Increasing the probability of project success**

Good management does not guarantee success, but it provides a systematic way to plan and control the project and therefore increases the likelihood of achieving its objectives.

Common Problems in SPM:

Software projects frequently experience difficulties because of the interaction between technical, organizational, human, and business factors.

- **Poor or unclear requirements**

If requirements are incomplete, ambiguous, inconsistent, or misunderstood, the development team may build the wrong system. For example, if a client says that an application must be "fast," the project team needs to understand what "fast" actually means in measurable terms.

Poor requirements can result in rework, increased cost, schedule delays, and customer dissatisfaction.

- **Changing requirements and scope creep**

Requirements may change because customers understand their needs better after seeing the software, business conditions change, or competitors introduce new features. **Scope creep** occurs when additional work or requirements are gradually added without appropriate evaluation and control.

Not every change is bad. The problem occurs when changes are introduced without considering their impact on **cost, schedule, resources, quality, and risk**.

- **Unrealistic estimation**

Project managers may underestimate effort, cost, or duration because of insufficient historical information, excessive optimism, lack of experience, or pressure to provide

an attractive deadline. Underestimation can result in overtime, reduced quality, missed deadlines, and cost overruns.

Overestimation can also be problematic because it may make a project unnecessarily expensive or cause an organization to reject a viable project.

- **Unrealistic deadlines**

Sometimes deadlines are imposed before the project has been properly estimated. When the required work cannot reasonably be completed within the available time, the team may experience excessive pressure, reduced testing, technical debt, poor quality, and employee burnout.

- **Human resource problems**

Software development depends heavily on skilled people. Problems may arise because of:

- Lack of required skills
- Staff turnover
- Poor team coordination
- Lack of motivation
- Communication problems
- Conflicts between team members
- Incorrect allocation of responsibilities

A single key employee leaving a project can sometimes have a significant effect on schedule and knowledge continuity.

- **Communication problems**

Miscommunication between customers, managers, analysts, developers, and testers can result in incorrect assumptions and rework. Effective communication is therefore essential throughout the project rather than only during project initiation.

- **Technical difficulties**

New technologies, integration problems, performance issues, security requirements, legacy systems, and unexpected technical constraints can affect project progress. Technical risks are particularly significant when the team has limited experience with the selected technology.

- **Poor quality management**

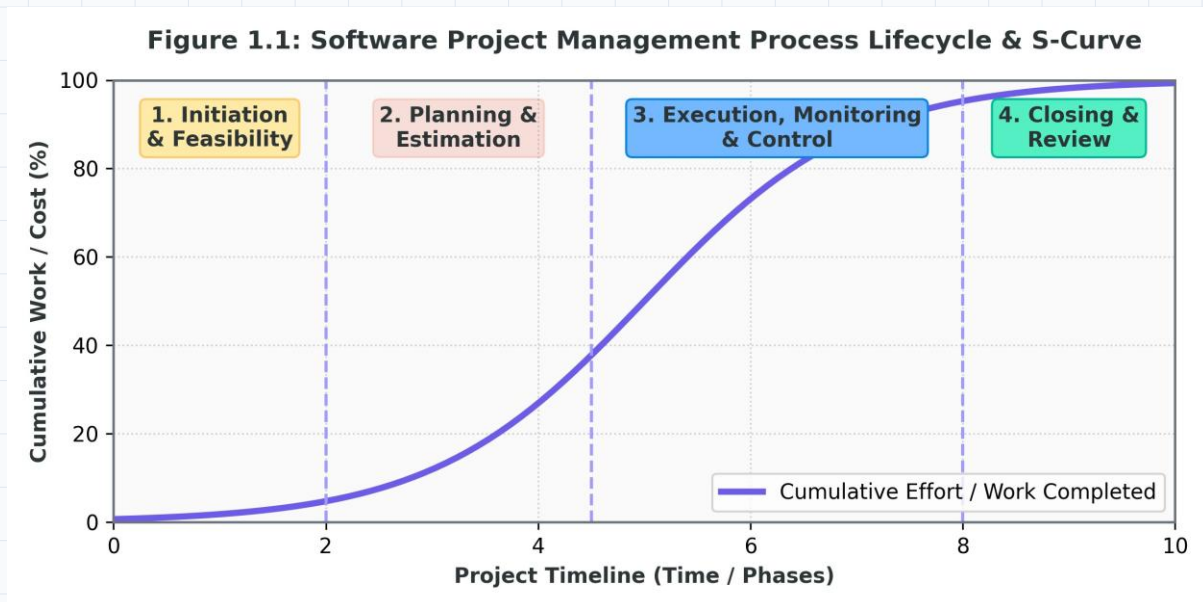
If quality is considered only at the end of development, defects may be discovered late when they are more expensive to correct. Quality should therefore be considered throughout planning, design, development, testing, and deployment.

- **Poor risk management**

Ignoring potential risks does not make them disappear. If risks are not identified and monitored, the project may be forced to react after the problem occurs. Effective project management attempts to identify important risks early and prepare appropriate responses.

1.4 SPM Process & Project Lifecycle

The software project management process spans five fundamental management process groups: Initiating (feasibility & charter), Planning (WBS, schedule, budget), Executing (development), Monitoring & Controlling (tracking progress & variance), and Closing (client handover & postmortem).



Relationship Between SPM Activities

The activities of software project management are strongly interconnected. For example, **estimation** affects the project schedule because the estimated effort influences how long activities may take. The **schedule** affects **cost** because resources must be paid for the time they are required. **Risk** can affect both cost and schedule. Changes in requirements can affect scope, effort, schedule, cost, and quality. Therefore, a project manager must manage the project as an interconnected system rather than treating each management activity independently.

1.5 Characteristics of a Good & Successful Software Project Manager

A software project manager is responsible for coordinating people, processes, resources, technical activities, stakeholders, risks, schedules, and project objectives.

A successful software project manager therefore requires a combination of **technical understanding, management ability, leadership, communication, decision-making, and interpersonal skills**.

1.5.1 Technical Competence

A project manager does not necessarily need to be the best programmer on the team, but should understand important technical concepts well enough to communicate with technical staff and make informed decisions.

Technical understanding helps the manager evaluate:

- Technical feasibility
- Architecture choices
- Technology risks
- Integration problems
- Development constraints
- Quality issues

The manager should understand technology without unnecessarily micromanaging technical specialists.

1.5.2 Planning and Organizing Ability

A good project manager must convert a broad project objective into manageable activities.

This requires the ability to:

- Define scope
- Develop WBS
- Estimate effort
- Create schedules
- Allocate resources
- Establish milestones
- Coordinate dependencies

Good organization reduces confusion and makes project progress easier to monitor.

1.5.3 Leadership

Leadership is more than assigning tasks. A project manager must provide direction, create trust, motivate team members, resolve conflicts, and maintain a positive working environment. A technically strong team can still fail if there is poor leadership and coordination.

1.5.4 Communication Skills

Communication is one of the most important skills of a project manager.

The manager communicates with:

- Customers
- Senior management
- Developers
- Testers
- Business analysts
- Vendors
- Other stakeholders

The project manager must be able to explain technical issues to non-technical stakeholders and business requirements to technical teams.

1.5.5 Decision-Making Ability

Projects frequently require decisions involving time, cost, quality, scope, resources, and risk. A good project manager gathers relevant information, evaluates alternatives, considers consequences, and makes decisions at the appropriate time.

Delaying every difficult decision can itself become a project risk.

1.5.6 Risk Management Ability

A successful project manager does not wait for problems to occur. The manager identifies potential risks, evaluates their probability and impact, develops response

strategies, and monitors them throughout the project. This proactive approach allows the team to prepare for uncertainty.

1.5.7 Problem-Solving Ability

Software projects inevitably encounter problems. A project manager should be able to determine:

What happened? → Why did it happen? → What are the possible solutions? → What are the consequences of each solution? → Which action should be taken?

The focus should be on solving the underlying problem rather than simply treating its symptoms.

1.5.8 Time Management

The project manager must understand priorities and ensure that important activities are completed on time. This includes monitoring milestones, dependencies, critical activities, deadlines, and schedule deviations.

1.5.9 Team Management

Software development is a team activity. A project manager must understand team capabilities, allocate responsibilities appropriately, encourage collaboration, and handle conflicts constructively. The manager should create an environment where team members can communicate problems without fear.

1.5.10 Negotiation Skills

Conflicts may occur between customers, management, and technical teams. For example, a customer may want additional features while management wants the project completed within the original deadline.

The project manager must negotiate realistic solutions by considering project constraints rather than simply accepting every demand.

1.5.11 Adaptability

Software projects operate in environments where requirements, technologies, business priorities, and risks can change. A successful project manager should be flexible enough to adapt plans when necessary while still maintaining appropriate control over scope and resources.

1.5.12 Ethical Responsibility

A project manager has responsibility toward the organization, customer, team, and other stakeholders.

Ethical behavior includes being honest about:

- Project status
- Costs
- Risks
- Schedule problems
- Quality issues
- Technical limitations

A manager should not hide serious problems simply to make project reports appear successful.

Good Project Manager vs. Successful Software Project Manager

A **good project manager** generally demonstrates sound management practices such as planning, communication, organization, and control.

A **successful software project manager** goes further by combining these management skills with technical understanding, leadership, adaptability, risk awareness, stakeholder management, and the ability to achieve project objectives in a changing software environment.

In other words, success is not simply about keeping a project schedule updated. It involves balancing scope, time, cost, quality, people, technology, risks, and stakeholder expectations to achieve a useful and acceptable software product.

Laughing Through the Chaos - Unit 1 Reality Check

By the time you finish this unit you will understand why software project management exists... and why so many project managers look like they haven't slept since 2019. A project, according to the big serious books, is a temporary thing with a start, an end, limited money, and a unique result. In software that "unique result" usually means the client will change their mind sixteen times and still act surprised when the deadline explodes. Software is invisible, complex, flexible, and somehow allergic to staying the same for more than three days. Building a bridge is easy by comparison: once the concrete is poured, nobody walks up and says, "Can we make it float and also change colour depending on the mood of the CEO?" In software that conversation happens every Tuesday.

This is why the same project that looked simple on day one suddenly needs three extra developers, a time machine, and a priest. Requirements start clear and then slowly mutate into something that looks like a wishlist written by a committee of caffeinated squirrels. Scope creep doesn't knock on the door; it slides in wearing socks and starts living in the living room. Estimation is pure comedy: someone says "three days," the universe laughs, and six weeks later the same person is still saying "almost done" with the confidence of a man who has never been almost done in his life. Deadlines are often decided before anyone has counted the work, which is like promising to finish a novel before deciding what the story is about.

A good software project manager is the poor soul standing in the middle of this circus trying to keep the elephants from eating the budget. They need just enough technical knowledge to know when the developers are lying, enough leadership to stop the team

from fighting over the coffee machine, and enough honesty to tell the boss that the project is currently on fire without getting fired themselves. Communication becomes an Olympic sport: explaining to the client why "make it faster" is not a requirement, explaining to the developers why the client is not actually evil, and explaining to senior management why the status report is not pure fiction.

In the end, software project management is less about perfect plans and more about surviving the beautiful chaos of building something that doesn't exist yet, with people who change their minds, technology that misbehaves, and a calendar that refuses to cooperate. If you can laugh while reading this, congratulations: you already understand the subject better than half the industry.

UNIT 2: Software Project Scheduling

Software project scheduling is the process of determining what activities need to be performed, when they should be performed, how long they will take, what resources are required, and the order in which they should be completed.

A project schedule provides a time-based plan for completing the project. It helps the project manager coordinate team members, allocate resources, monitor progress, identify delays, and determine the expected project completion date.

Objectives of Project Scheduling

The main objectives are:

- Identify all activities required to complete the project.
- Determine the sequence and dependencies between activities.
- Estimate the duration of each activity.
- Allocate appropriate resources.
- Establish milestones and deadlines.
- Identify critical activities.
- Determine the expected project completion date.
- Monitor and control project progress.
- Identify possible delays and scheduling risks.

2.1 Objectives of Activity Planning

Activity planning is the process of identifying the activities required to complete a software project, determining their sequence and dependencies, estimating their duration, and assigning the required resources. It converts the overall project objectives into a set of manageable activities that can be scheduled and monitored.

The main objectives of activity planning are:

1. **Identifying Activities:** Determines all the major tasks required to complete the project, from requirements analysis through development, testing, deployment, and closure.
2. **Feasibility Determination:** Determines whether the project can realistically be completed within the available time, resources, and constraints.
3. **Resource Allocation:** Identifies the people, tools, equipment, and other resources required for different activities and helps avoid resource conflicts.
4. **Cost Estimation:** Helps refine project cost estimates by identifying the work and resources involved.
5. **Milestone Definition:** Defines important points in the project that can be used to measure progress and determine whether the project is following the planned schedule.
6. **Dependency Identification:** Determines the logical relationships between activities and the order in which they must be performed.
7. **Critical Activity Identification:** Helps identify activities that have little or no flexibility in their schedule and therefore can directly affect the project completion date. Proper activity planning provides the foundation for preparing a realistic project schedule and helps the project manager identify possible delays before they occur.
8. **Estimating Time and Effort:** Provides a basis for estimating how much time and effort each activity will require.

2.2 Work Breakdown Structure (WBS)

A **Work Breakdown Structure (WBS)** is a hierarchical decomposition of the total project work into smaller and more manageable components. It starts with the overall project and progressively divides it into major deliverables, sub-deliverables, activities, and finally work packages.

The purpose of WBS is to make a large and complex software project easier to understand, estimate, assign, schedule, and control. It focuses on **what work needs to be completed**, rather than describing exactly how the work will be performed.

For example, a software project may be divided into Requirements, Design, Development, Testing, and Deployment. Each of these can then be further divided into smaller components and tasks.

DEFINITION: WBS Rules & Work Packages

- **100% Rule:** The WBS should contain all the work required to achieve the defined project scope. No required work should be left out, and unrelated work should not be included.
- **Mutual Exclusion:** Each WBS element should represent distinct work so that the same work is not counted under more than one element.
- **Work Package:** A work package is the lowest level of the WBS at which work can be estimated, scheduled, assigned to a responsible person or team, and monitored.

WBS Decomposition Styles:

- **Product-Oriented WBS:** Organizes work according to the major components or deliverables of the software, such as User Interface, Application Server, and Database.
- **Process-Oriented WBS:** Organizes work according to development activities such as Requirements, Design, Coding, Testing, and Deployment.
- **Hybrid WBS:** Combines both approaches, using product components at one level and development activities at another level.

A well-defined WBS helps improve **effort estimation, resource allocation, scheduling, cost estimation, and progress tracking** because the project is divided into smaller units that are easier to manage.

2.3 Network Planning Models & Dependencies

A **network planning model** is a graphical representation of project activities and the logical relationships between them. It shows the sequence in which activities are performed, identifies activities that can be performed in parallel, and illustrates how one activity depends on another. Network planning is useful for developing the project schedule, estimating project duration, identifying the critical path, calculating activity dates, and understanding how delays in one activity may affect other activities and the overall project.

2.3.1 Network Diagram

A **network diagram** is a visual representation of the activities of a project and the dependencies between those activities. In a typical network diagram, **nodes represent activities**, while **arrows or connecting lines represent the logical relationships between activities**. Each activity can be associated with information such as its duration, start and finish dates, and resources. By arranging activities according to their dependencies, the network diagram provides a clear picture of how the project work flows from the beginning to completion.

For example, a simple software project may follow the sequence:

Requirements → Design → Coding → Testing → Deployment

However, some activities may occur in parallel. For example, documentation may begin while development is in progress. A network diagram makes these relationships easier to understand and provides the foundation for techniques such as **CPM** and **PERT**.

Steps for Creating a Network Diagram

A network diagram is generally developed through the following process:

1. **Identify the activities:** List all activities required to complete the project, usually from the WBS.

2. **Determine dependencies:** Identify which activities must be completed or started before other activities can begin.
3. **Determine activity durations:** Estimate the time required for each activity.
4. **Arrange activities in logical order:** Place activities according to their dependencies and identify activities that can occur in parallel.
5. **Connect the activities:** Use arrows or connecting lines to represent the relationships between activities.
6. **Review the network:** Check that all activities and dependencies are correctly represented and that there are no unnecessary or missing relationships.
7. **Analyze the network:** Use the completed network as a basis for calculating project duration, float, and the critical path using methods such as CPM or PERT.

Types of Network Diagram

Two commonly discussed approaches to project network diagrams are **Activity-on-Arrow (AOA)** and **Activity-on-Node (AON)**.

Activity-on-Arrow (AOA), also called the **Arrow Diagramming Method (ADM)**, represents activities using arrows, while nodes represent the beginning or end of activities. AOA primarily represents Finish-to-Start relationships and may require **dummy activities** to represent certain dependencies. It is an older approach and is less commonly used in modern project scheduling.

Activity-on-Node (AON), commonly implemented through the **Precedence Diagramming Method (PDM)**, represents activities as nodes and uses arrows to show the relationships between them. PDM is widely used because it can represent four types of activity dependencies: Finish-to-Start, Start-to-Start, Finish-to-Finish, and Start-to-Finish. It can also represent lead and lag times more conveniently.

How to Draw Arrow Diagramming Method (ADM)?

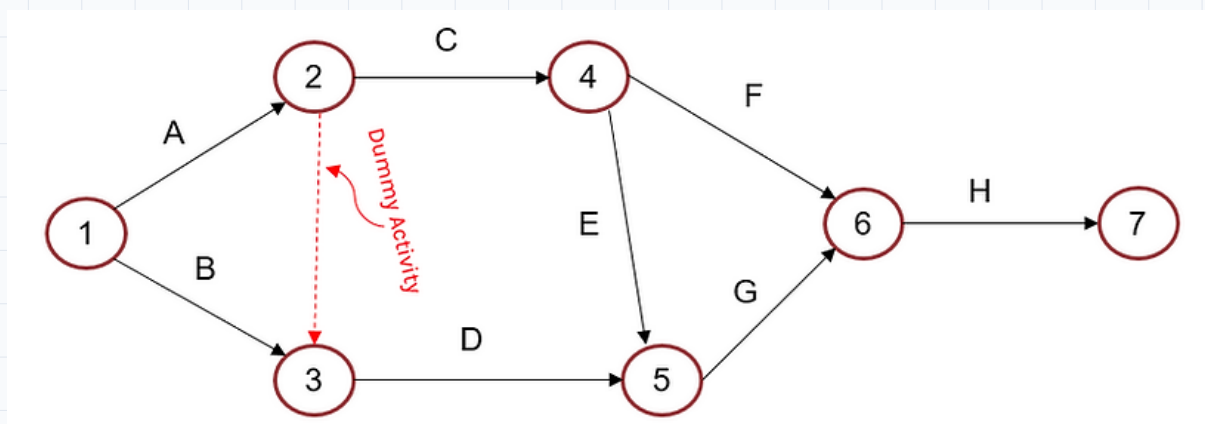


Fuad Abutaha
Nov 23, 2022 · 1 min read

How to Draw Arrow Diagramming Method (ADM)

Updated: Apr 9, 2025

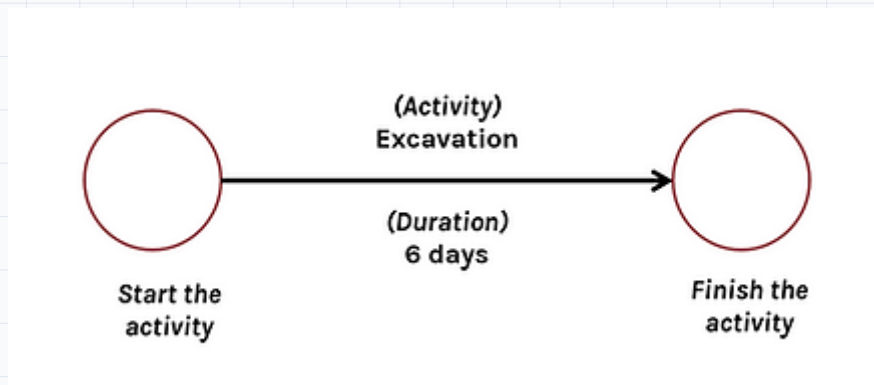
How to Draw ADM is copied from here Link: <https://www.skillsitc.com/post/how-to-draw-arrow-diagramming-method-adm>



- In the Arrow Diagramming Method (ADM), **activities are represented by arrows**, and **nodes (circles or boxes)** represent **events** (start or finish points). Also called Activity-on-Arrow (AOA) network diagrams.
- The start and end of each node or event are connected to an arrow. Between the two nodes lies an arrow that represents the activity. Nodes or circles are the starting and ending points of activities.
- Each arrow shows a **task**, and the connection points (nodes) represent the **precedence relationships**.
- Dummy activities (dashed arrows) are sometimes used to show logical relationships that don't require time.

- Can only show finish-to-start dependencies.

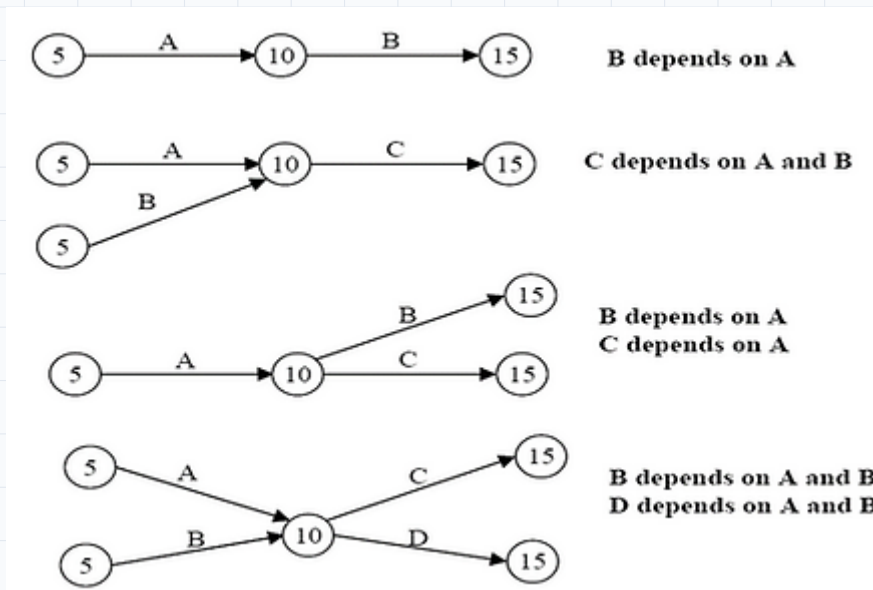
Single Activity on AOA network diagram



Connection of Two Activities

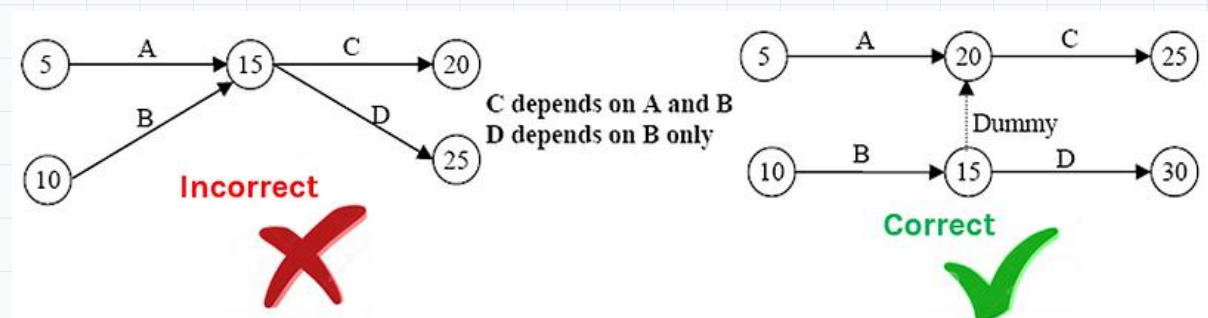


Examples of different dependencies



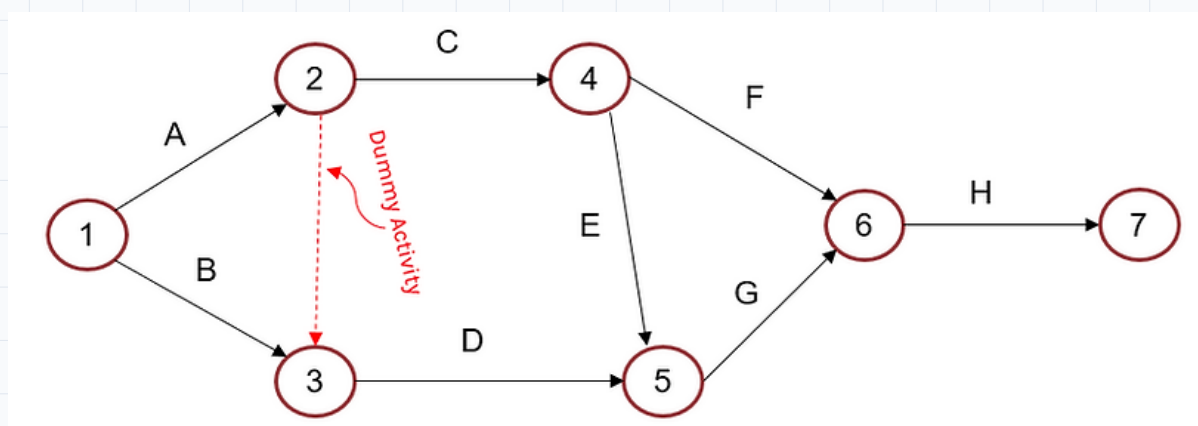
Dummy activity

Zero duration, consumes no resources, drawn as dashed lines, and used to adjust the network diagram



Example 1

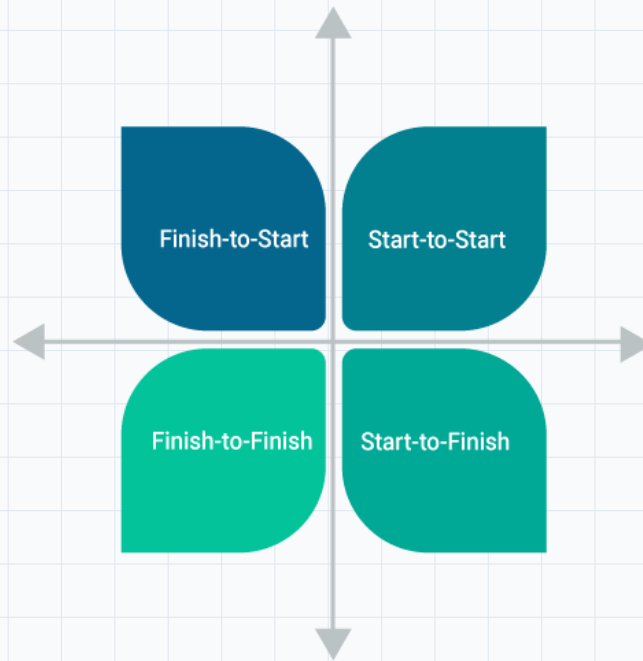
Activity	Immediate predecessors
A	-
B	-
C	A
D	A, B
E	C
F	C
G	D, E
H	F, G



Precedence Diagramming Method (PDM / AON) Dependency Types:

The **Precedence Diagramming Method (PDM)**, also called **Activity-on-Node (AON)**, represents activities as nodes and uses connecting lines to show their dependencies.

The main dependency relationships are:



- **Finish-to-Start (FS):** The successor activity cannot start until the predecessor activity finishes. *Example: Testing starts after coding finishes.*
- **Start-to-Start (SS):** The successor activity cannot start until the predecessor activity starts. *Example: Documentation may start after development begins.*
- **Finish-to-Finish (FF):** The successor activity cannot finish until the predecessor activity finishes.
- **Start-to-Finish (SF):** The successor activity cannot finish until the predecessor activity starts. This relationship is less commonly used.
- **Lag Time:** A required delay between two related activities. *Example: A task may start 3 days after its predecessor finishes.*
- **Lead Time:** Allows a successor activity to begin earlier than it would under the normal dependency relationship.

Understanding dependencies is important because changing the timing of one activity can affect several connected activities and, in some cases, the entire project schedule.

Types of Dependencies in Precedence Diagramming Method

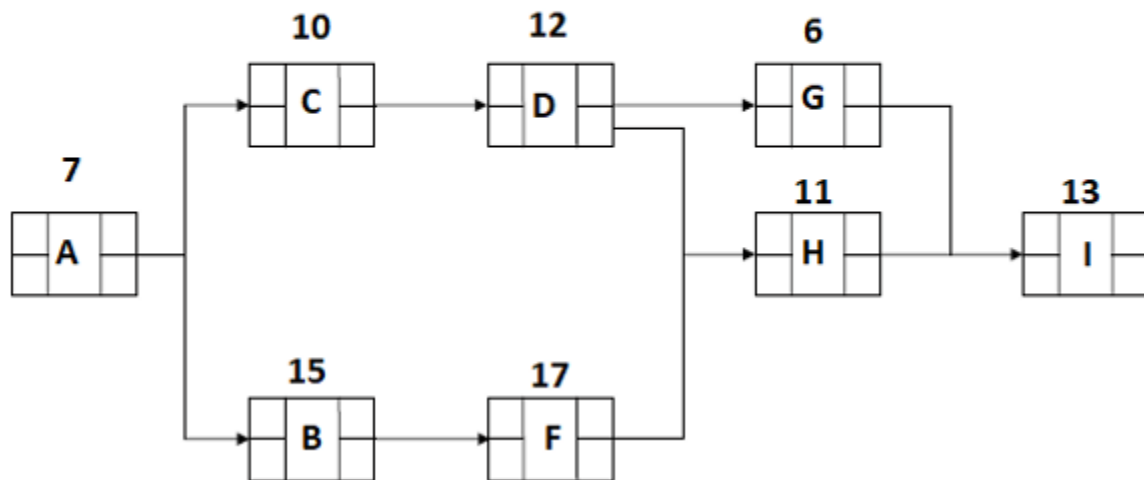
- **Mandatory Dependency** or hard logic is an integral part of the work. For instance, if you're working on a project, you cannot test a screen that is yet to be developed entirely in the project. Consequently, in this case, there is a mandatory dependency between the development and testing of the screen.
- **Discretionary Dependency** also referred to as preferential or soft logic is controlled by the project team and can be changed to abbreviate the project. In this technique, activities stay the same, yet, the order changes.
- **External Dependency** comes from outside of the project — for instance, a government regulation expected to finish before proceeding with the outstanding project activities.
- **Internal dependency** is a method that involves a precedence relationship between project activities. For instance, if there are activities appointed to a member of the project team, he can't begin to work on that until the previous project finishes even though there is no mandatory dependency.

Example of Precedence Diagramming Method (PDM)

Example of PDM and types of dependencies in PDM is copied from link:
<https://www.invensislearning.com/blog/precedence-diagramming-method/>

A visual representation technique prepares the project schedule network diagrams, here is a precedence diagramming method example that analyzes the basics and will help you understand the whole process.

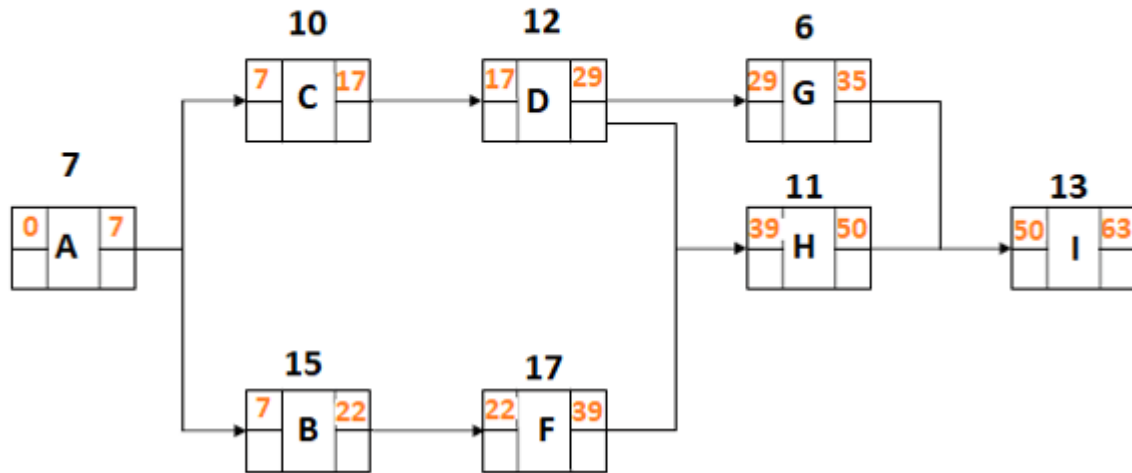
PRECEDENCE DIAGRAM METHOD



In the example mentioned above, activities are A, B, C, D, F, G, H, and I. Durations are the numbers above the boxes. Activity B is a predecessor activity that logically comes before a dependent activity F in this network system. Activity F is a successor activity A dependent activity that inevitably occurs after Activity F. There is an F-S Relationship between activity B and F.

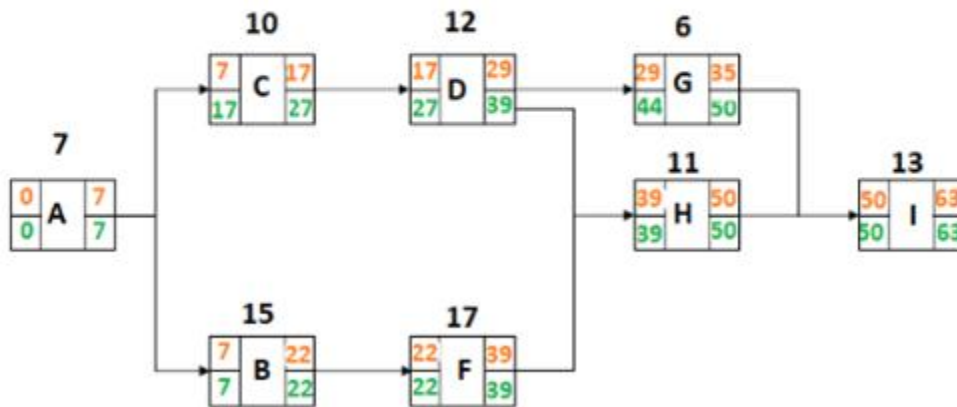
Step 1: PDM Forward Pass Calculation specifies the minimum dates at which each activity can be performed and, ultimately, the minimum duration of a project.

FORWARD PASS CALCULATION



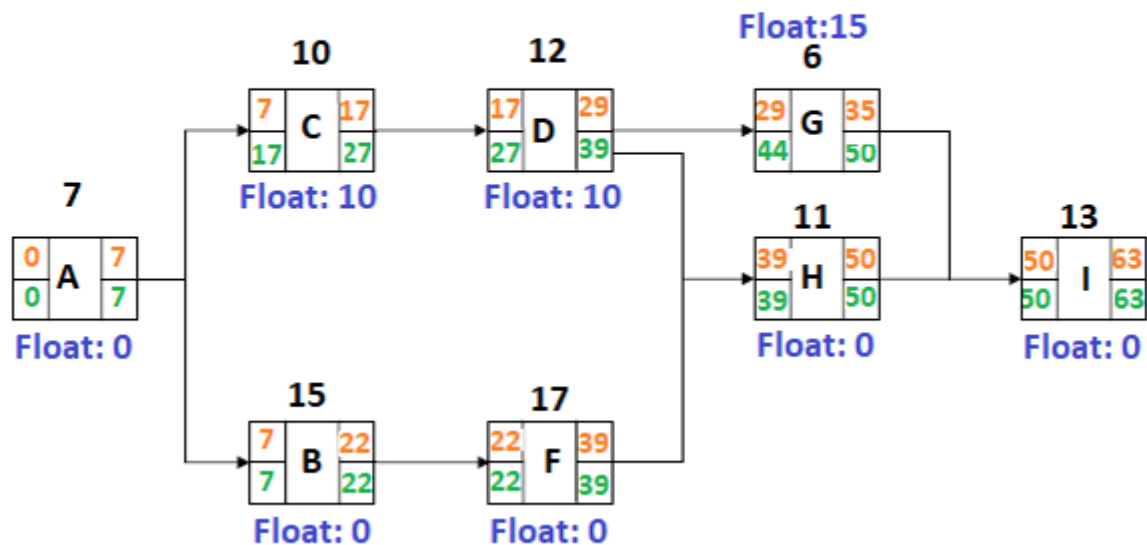
Step 2: PDM Backward Pass Calculation determines the latest dates by which each activity can be performed without increasing the project's minimum duration.

BACKWARD PASS CALCULATION



Step 3: PDM Float Calculation for Each Activity

FLOAT CALCULATION

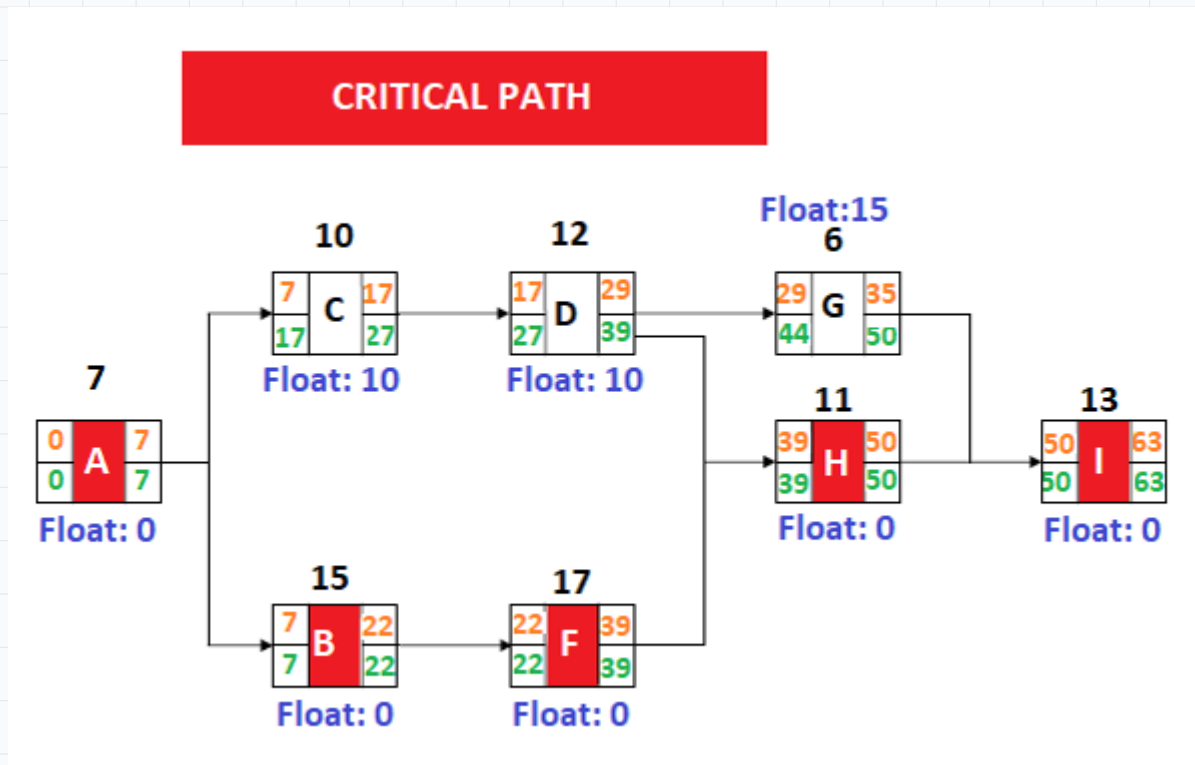


Total float is the amount of delay that an activity faces without delaying the project completion date. Float is 0 on the critical path.

- **Float:** $LS - ES = 17 - 7 = 10$
- **Total Float:** $LF - EF = 27 - 17 = 10$

You calculate the total float by subtracting the Early Start date of activity from its Late Start date. Or, the Early Finish date from its Late Finish date.

Step 4: PDM Identifying the Critical Path



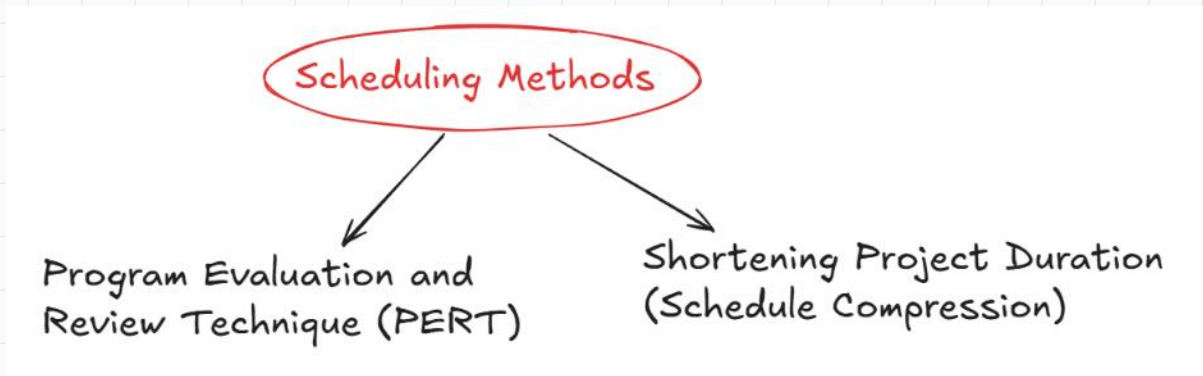
When we analyze the network diagram, we will see that there are some paths and every path has a duration.

The precedence diagramming method has its activity-on-node network and additional precedence relationships. It offers a simpler and increasingly adaptable system way to deal with demonstrating large projects. PDM enhances the project planner's ability to precisely demonstrate large projects. And, the clients ought to know about and be ready to adequately manage its highlights.

Network diagrams provide a clear visual understanding of the project's workflow and dependencies. They help project managers determine the sequence of activities, identify activities that can be performed simultaneously, estimate the overall project duration, and identify the **critical path**. They also help in analyzing the effect of delays and provide a foundation for applying **CPM** and **PERT**.

2.5 Scheduling Methods

The major scheduling methods used in software project management include:



2.5.1 Program Evaluation and Review Technique (PERT)

PERT (Program Evaluation and Review Technique) is a scheduling technique used when the duration of project activities is uncertain. Unlike CPM, which generally uses a single estimated duration, PERT uses **three time estimates** to provide a more realistic expected duration.

The three estimates are:

- o **Optimistic Time (a)**: The shortest reasonable time required when conditions are favorable.
- o **Most Likely Time (m)**: The time expected under normal working conditions.
- o **Pessimistic Time (b)**: The longest reasonable time expected under unfavorable conditions.

PERT gives greater importance to the **most likely estimate** while still considering optimistic and pessimistic conditions.

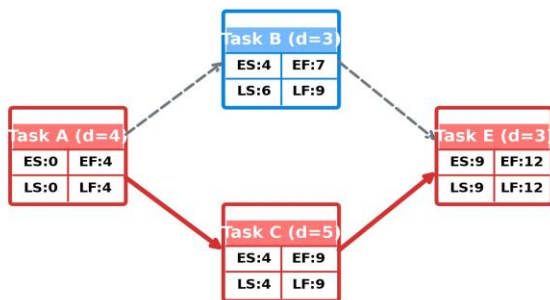
2.5.2 Critical Path Method (CPM) & Float Calculations

The **Critical Path Method (CPM)** is a scheduling technique used to determine the longest sequence of dependent activities in a project network. The duration of this longest path determines the **shortest possible time** in which the project can be completed.

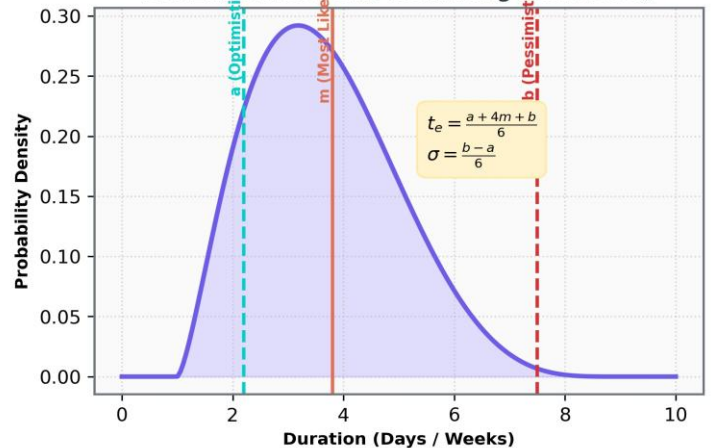
The activities on the critical path are especially important because they have **zero total float**. Therefore, a delay in a critical activity can delay the entire project unless the delay is recovered elsewhere.

Figure 2.1: Activity Planning Models (CPM Critical Path & PERT Distribution)

A. CPM Network & Critical Path (A-C-E)



B. PERT Three-Point Estimating (Beta Dist.)



CPM vs PERT

Feature	CPM	PERT
Full Form	Critical Path Method	Program Evaluation and Review Technique
Main Use	Scheduling and critical path	Scheduling under uncertainty

Time Estimates	Usually one	Three
Duration	More deterministic	Probabilistic
Main Focus	Time, critical path, float	Uncertainty and expected duration
Common Use	Well-defined projects	Projects with uncertain activity durations

2.6 Shortening Project Duration (Schedule Compression)

Schedule compression refers to reducing the planned project duration without reducing the project scope. It is normally considered when a project must meet an earlier deadline. Two common techniques are:

- o **Project Crashing:** Additional resources are assigned to critical activities to reduce their duration. This may involve adding developers, increasing working hours, or using additional resources. Crashing generally increases cost, so it should be applied where the reduction in project duration justifies the additional cost.
- o **Fast Tracking:** Activities that were originally planned to be performed sequentially are performed partly in parallel. For example, some testing may begin while development is still being completed. Fast tracking can reduce project duration but increases the possibility of errors, coordination problems, and rework.

Both techniques should be applied carefully because shortening the schedule may increase **cost, risk, or workload**.

Laughing Through the Chaos – Unit 2 Reality Check

Software project scheduling is the noble art of pretending you know exactly when everything will finish while secretly knowing the universe is laughing at your Gantt chart. In theory, you list every activity, figure out what depends on what, guess how long each will take, throw some humans at them, and announce a completion date with the confidence of a weather forecast. In reality, you create a Work Breakdown Structure that starts looking neat and hierarchical and ends up resembling a family tree after three generations of secrets. The 100% rule says everything must be included and nothing extra, which is adorable until the client casually adds "just one more tiny feature" that somehow needs its own continent in the WBS.

Then come the network diagrams, where we draw boxes and arrows like children who have discovered markers and a deadline. Activity-on-Arrow is the old-school version that needs dummy activities just to make the logic work, while Activity-on-Node (the modern favourite) lets you connect things with Finish-to-Start, Start-to-Start, and other relationships that sound romantic until you realise Finish-to-Start simply means "you cannot test the login page before someone actually builds the login page." Some dependencies are mandatory (you really cannot test what does not exist), some are discretionary (the team just prefers doing things in a certain order because it feels right), and some are external (the government still hasn't approved the API, so everyone sits and stares at the wall).

Once the diagram is drawn, we meet the critical path — the longest chain of activities that has zero float, meaning zero room for error, zero chance to breathe, and zero mercy. Delay anything on that path and the entire project slips like a banana peel under a boardroom table. PERT tries to be realistic by using three time estimates: optimistic

(everything goes perfectly and the coffee machine works), most likely (normal chaos), and pessimistic (the lead developer leaves, the server catches fire, and someone accidentally deploys to production on a Friday). The formula then produces a beautiful expected duration that still ends up wrong, but at least it was mathematically wrong.

When the deadline is suddenly moved forward because "the CEO has a conference," we enter schedule compression territory. Crashing means throwing more people and money at the critical activities until the budget starts crying. Fast tracking means doing things in parallel that were never meant to happen at the same time, which is like cooking the chicken while the oven is still being installed — possible, but the results are rarely pretty. In the end, a project schedule is less a precise plan and more a shared hallucination that everyone agrees to believe in until the first major delay arrives and the hallucination politely excuses itself.

UNIT 3: Software Estimation Techniques

3.1 Problems with Over and Under Estimation

Software effort estimation is the process of predicting the amount of effort, time, and resources required to develop a software system. Effort is commonly expressed in **Person-Months (PM)**, while duration represents the calendar time required to complete the project.

Accurate estimation is important because project schedules, budgets, staffing, and resource allocation are based on these estimates. Both under-estimation and over-estimation can create problems.

Estimation Error	Consequences & Pathologies
Under-Estimation (Estimating too low)	• Team burnout and high stress. • Quality degradation (skipping testing and code reviews). • Missed deadlines, client dissatisfaction, and cost overruns. • Brooks' Law: Adding manpower to a late software project makes it later.
Over-Estimation (Estimating too high)	• Parkinson's Law: Work expands so as to fill the time available for its completion. • Student Syndrome: Procrastination until the deadline approaches. • Bloated project budgets and loss of competitive bidding advantage.

3.2 Software Effort Estimation Techniques

Software effort estimation techniques are methods used to predict the effort, time, and resources required for a software project. No single technique is suitable for every project, so organizations may use different techniques and compare their results.

The major approaches include **Expert Judgment, Estimating by Analogy, Top-Down Estimating, Bottom-Up Estimating, Function Point Analysis, and COCOMO.**

1. Expert Judgment

In **Expert Judgment**, experienced software professionals estimate the required effort based on their knowledge, previous experience, and understanding of similar projects. Experts consider factors such as project complexity, team capability, technology, requirements, and previous project experience.

The method is useful when historical data is limited, but the estimate can be affected by **personal bias, optimism, or differences in experience.** Techniques such as group discussion or Delphi-based estimation can be used to reduce individual bias.

2. Estimating by Analogy

Estimating by Analogy involves comparing the proposed project with one or more previously completed projects that are similar in size, functionality, technology, or domain.

For example, if a company previously developed a similar banking application and knows the effort required, that project can be used as a reference for estimating a new banking application.

The accuracy of this method depends heavily on how closely the previous project resembles the new project and on the quality of historical project data.

3. Top-Down Estimating

In the **Top-Down approach**, the total effort or cost of the project is estimated first. The estimated total is then divided among major project phases or components.

For example:

Total Project Effort → Requirements → Design → Coding → Testing → Deployment

This approach is useful during the early stages when detailed project information is not yet available. However, it may overlook the effort required for smaller individual activities.

4. Bottom-Up Estimating

In the **Bottom-Up approach**, the project is divided into small tasks or work packages, and the effort required for each is estimated separately. The individual estimates are then added to obtain the total project estimate.

Individual Work Package Estimates → Sum of Estimates → Total Project Effort

This approach can provide a more detailed and realistic estimate because the work is considered at a lower level. However, it requires a reasonably complete **WBS** and can take more time to prepare.

Top-Down vs. Bottom-Up Estimating

Basis	Top-Down	Bottom-Up
Starting point	Entire project	Individual tasks/work packages
Detail required	Less	More
Suitable stage	Early project stages	When detailed requirements are available
Accuracy	Generally lower when details are unknown	Generally more detailed
Main limitation	May overlook individual task effort	Time-consuming and requires detailed WBS

2. Parametric / Algorithmic Models:

A. Function Point Analysis (Albrecht FP Model):

Function Point Analysis (FPA) is a software sizing and estimation technique that measures software based on the **functionality provided to the user** rather than directly counting lines of source code.

It is useful because the same functionality can require different amounts of source code depending on the programming language and technology used.

KEY FORMULA & METHOD: Function Point Calculation Steps

1. **Count 5 Functional Components:** External Inputs (EI), External Outputs (EO), External Inquiries (EQ), Internal Logical Files (ILF), External Interface Files (EIF).

2. **Calculate Unadjusted Function Points (UFP):**

$$UFP = \sum_{i=1}^5 \sum_{j=1}^3 (Count_{ij} \times Weight_{ij})$$

3. **Calculate Value Adjustment Factor (VAF):** Based on 14 General System Characteristics

$$VAF = 0.65 + 0.01 \times \sum_{i=1}^{14} C_i \quad (\text{Range: 0.65 to 1.35})$$

3. **Final Function Points (FP):** $FP = UFP \times VAF$

B. COCOMO Model (Constructive Cost Model - Barry Boehm):

COCOMO (Constructive Cost Model) is a software cost and effort estimation model developed by **Barry Boehm**. The Basic COCOMO model estimates the effort required to develop software based primarily on its size, expressed in **Kilo Source Lines of Code (KSLOC)**.

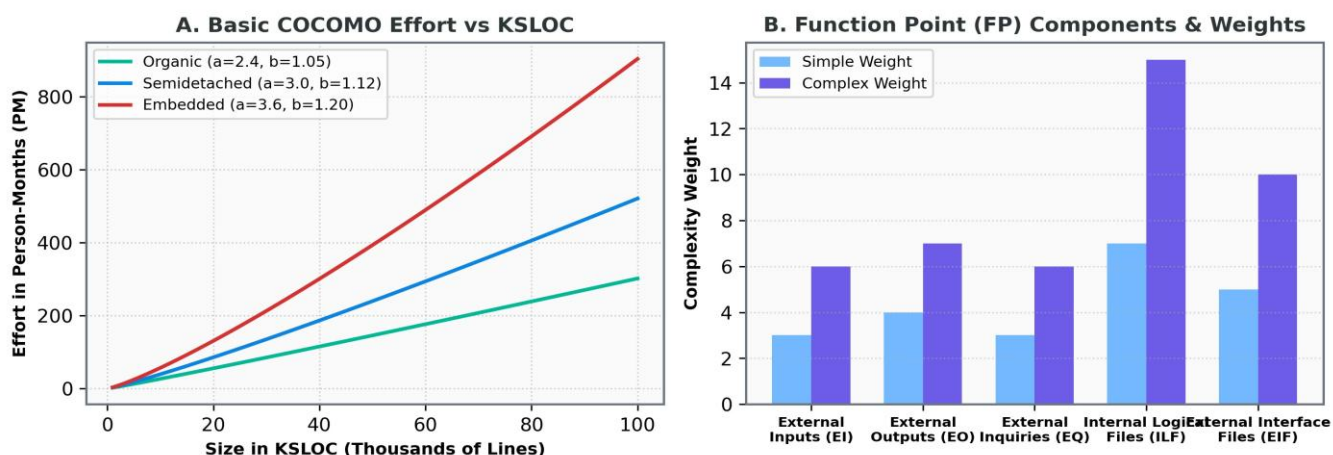
Basic *COCOMO* classifies projects into three development modes based on their size, complexity, team experience, and environmental constraints.

COCOMO Mode	Project Complexity	Effort Equation (PM)	Duration Equation (Tdev)
Organic	Small team, familiar domain, flexible requirements	$PM = 2.4 \times (KSLOC)^{1.05}$	$Tdev = 2.5 \times (PM)^{0.38}$
Semidetached	Medium scale, mixed experience, rigid/flexible mix	$PM = 3.0 \times (KSLOC)^{1.1}$	$Tdev = 2.5 \times (PM)^{0.35}$
Embedded	Complex, tight hardware/software constraints	$PM = 3.6 \times (KSLOC)^{1.20}$	$Tdev = 2.5 \times (PM)^{0.32}$

Where:

- **PM** = Effort in Person-Months
- **KSLOC** = Thousand Source Lines of Code
- **Tdev** = Development time in months

The basic *COCOMO* model is useful for obtaining an early estimate of project effort and development time. However, its accuracy depends on the quality of the software size estimate and how well the selected project mode represents the actual project.

Figure 3.1: Parametric Software Estimation (COCOMO & Function Point Analysis)

Summary of Estimation Technique

Technique	Main Basis	Major Advantage	Main Limitation
Expert Judgment	Experience and knowledge	Simple and useful when data is limited	Subjective and may contain bias
Analogy	Similar completed projects	Uses real historical experience	Depends on project similarity
Top-Down	Overall project estimate	Quick and useful early	May overlook detailed task effort
Bottom-Up	Individual work packages	Detailed and easier to justify	Requires detailed WBS
Function Point Analysis	User functionality	Independent of programming language	Requires proper function classification

COCOMO	Software size and project mode	Provides systematic effort and time estimates	Depends on accurate size and suitable model selection
---------------	--------------------------------	---	---

Laughing Through the Chaos - Unit 3 Reality Check

Software estimation is the only profession where you can be wrong in both directions and still somehow make everything worse. Under-estimate and the team burns out, quality dies, deadlines explode, and Brooks' Law arrives like an uninvited relative: adding more people to a late project just makes it later and more expensive. Over-estimate and Parkinson's Law kicks in — work magically expands to fill every available hour — while Student Syndrome quietly ensures nobody starts anything serious until the panic sets in two days before the deadline.

In practice, estimation usually begins with Expert Judgment, which is basically a senior developer closing their eyes, remembering the last disaster, and saying a number that feels emotionally correct. Estimating by Analogy is slightly more scientific: "Last time we built something vaguely similar it took eight months, so this one should also take eight months... ignoring the fact that the previous one nearly killed us." Top-Down starts with a big round number and divides it like a parent cutting a cake for arguing children. Bottom-Up goes the opposite way — estimate every tiny task and then watch the total become a number so large that management pretends they never saw it.

When numbers still refuse to behave, we bring in Function Points, where we count screens, reports, and files as if software were a collection of Lego pieces, then multiply by a mysterious adjustment factor that somehow always lands between 0.65 and 1.35. And then there is COCOMO, the model that looks at your project and classifies it as Organic (small, friendly, everyone knows each other), Semidetached (medium chaos), or Embedded (the kind of project where the hardware, the software, and the client's expectations are all

actively trying to kill you). The formulas spit out Person-Months with impressive confidence, yet somehow the actual project still finishes late, over budget, and with three developers quietly updating their LinkedIn.

In the end, every estimation technique is just a different way of making peace with the same uncomfortable truth: we are trying to predict the future of something that doesn't exist yet, using data from projects that also refused to behave.

UNIT 4: Software Evaluation and Costing

4.1 Project Evaluation: Strategic & Technical Assessment

Before committing financial capital to a software project, organizations conduct feasibility evaluation across multiple dimensions

4.1 Project Evaluation

Project evaluation is the process of examining a proposed software project before committing significant time, money, and resources to it. The purpose is to determine whether the project is **worth undertaking, technically feasible, financially beneficial, and consistent with the organization's objectives.**

A software project may appear attractive because of its expected benefits, but it may still fail if the organization does not have the required technology, skills, resources, or financial capacity. Therefore, project evaluation considers several aspects before the final decision is made.

4.1.1 Strategic Assessment

Strategic assessment determines whether a proposed software project supports the organization's overall business strategy and long-term objectives. The project should contribute to the direction in which the organization wants to develop rather than being selected only because the technology is available.

Strategic assessment generally considers:

- **Organizational objectives:** Whether the project supports important business goals.
- **Market position:** Whether the project can improve the organization's position in the market.

- **Competitive advantage:** Whether the software can provide benefits that competitors may not easily achieve.
- **Business opportunities:** Whether the project can create new products, services, customers, or sources of revenue.
- **SWOT factors:** The project's strengths, weaknesses, opportunities, and threats may also be considered.

A project with strong strategic alignment is more likely to receive organizational support because its expected benefits are connected with long-term business priorities.

4.1.2 Technical Assessment

Technical assessment determines whether the proposed software can be developed and operated successfully using the available technology, infrastructure, and technical skills.

The assessment considers the proposed architecture, hardware and software requirements, compatibility with existing systems, availability of skilled personnel, performance requirements, security requirements, and the maturity of the selected technology.

For example, an organization may identify a valuable business opportunity but may not have developers with the required technical skills or infrastructure. In such a situation, additional training, recruitment, outsourcing, or technology changes may be necessary before the project can be undertaken.

Therefore, technical assessment helps identify technical limitations and uncertainties before they become major project problems.

4.2 Cost-Benefit Analysis

Cost-Benefit Analysis (CBA) is a technique used to compare the expected costs of a software project with its expected benefits. It helps management determine whether the benefits obtained from the project are sufficient to justify the investment.

The costs of a software project are not limited to development expenses. They may include salaries, hardware, software licenses, training, installation, maintenance, support, and operational expenses. Similarly, benefits may include increased revenue, reduced operating costs, improved productivity, better customer service, and other strategic advantages.

The basic idea is:

Project Benefits - Project Costs = Overall Economic Value.

Cost-benefit analysis is particularly useful when an organization has several possible projects and limited financial resources. It provides a basis for comparing alternatives and selecting projects that provide greater value.

However, not every benefit can be expressed easily in monetary terms. Improved customer satisfaction, better decision-making, improved security, or competitive advantage may be difficult to assign an exact financial value. Therefore, financial analysis should be considered together with strategic, technical, and risk factors.

4.3 Cash Flow Forecasting

Cash flow forecasting is the process of estimating the amount and timing of cash entering and leaving a project during its lifetime. It is important because software projects usually require significant expenditure during development, while the expected financial benefits may be received only after the system becomes operational.

Cash inflows may come from additional sales, increased revenue, cost savings, or other financial benefits generated by the system. Cash outflows include development costs, employee salaries, infrastructure, training, maintenance, licenses, and operational expenses.

The basic relationship is:

Net Cash Flow = Cash Inflows – Cash Outflows.

A cash-flow forecast allows management to see when money will be spent, when benefits are expected, and whether the project is financially sustainable. It also provides the cash-flow information required for financial evaluation techniques such as Payback Period and NPV.

4.4 Cost-Benefit Evaluation Techniques

Several techniques can be used to evaluate the financial attractiveness of a software project. The important techniques include Payback Period, Return on Investment (ROI), and Net Present Value (NPV).

4.4.1 Payback Period

The Payback Period is the amount of time required for the project to recover its initial investment from the net cash inflows generated by the project. If the annual net cash inflow is approximately constant:

Payback Period = Initial Investment / Annual Net Cash Inflow

For example, if a project requires an initial investment of Rs. 5,00,000 and generates Rs. 1,00,000 of net cash inflow per year, its payback period is approximately 5 years.

A project with a shorter payback period generally recovers its investment faster. However, payback does not consider the value of cash flows received after the investment has been recovered and, in its basic form, does not consider the time value of money.

4.4.2 Return on Investment (ROI)

Return on Investment (ROI) measures the profitability of an investment in relation to the amount invested. It provides a percentage that indicates how much return is obtained from the investment.

First, annual profit can be calculated as:

$$\text{Annual Profit} = (\text{Total Income} - \text{Total Expenses}) / \text{Project Duration (Years)}$$

Then:

$$\text{ROI} = (\text{Annual Profit} / \text{Total Investment}) \times 100\%$$

A higher ROI generally indicates a more attractive financial return, assuming the projects being compared have similar conditions and risks.

4.4.3 Net Present Value (NPV)

Net Present Value (NPV) evaluates a project by considering the time value of money.

The basic idea is that money received in the future is worth less than the same amount of money received today.

Future cash flows are therefore converted into their present value using a discount rate.

$$\text{NPV} = \sum_{t=1}^n \frac{C_t}{(1+r)^t} - C_0$$

Where:

C_t = Net cash flow in year t

r = Discount rate

t = Time period

C_0 = Initial investment

The interpretation is:

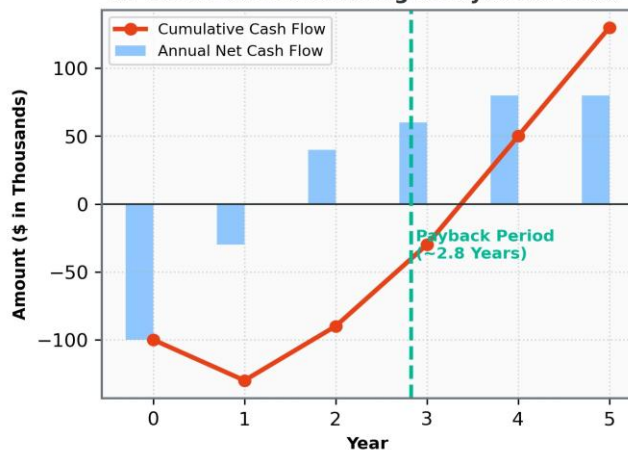
• **NPV > 0**: The project is expected to generate value above the required return.

• **NPV = 0**: The project is expected to earn approximately the required rate of return.

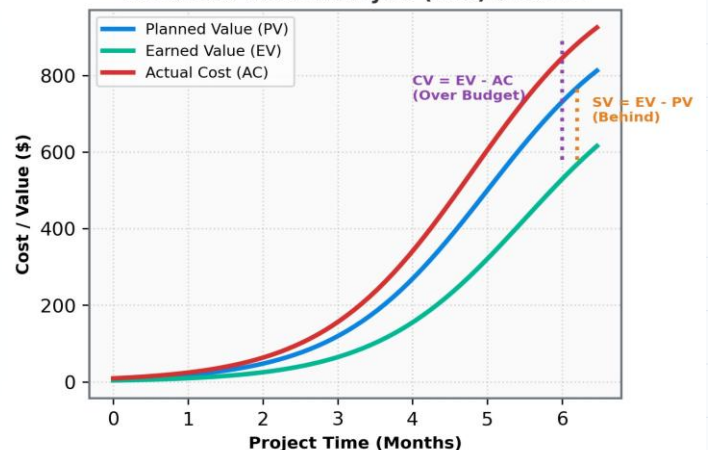
• **NPV < 0**: The project does not generate sufficient return at the given discount rate.

When comparing alternative projects, a **higher positive NPV** is generally preferred because it indicates greater financial value, assuming other important factors are acceptable.

A. Cash Flow Forecasting & Payback Period



B. Earned Value Analysis (EVA) S-Curve



4.5 Risk Evaluation

Financial calculations alone are not sufficient for selecting a software project because actual project results may differ from estimates. **Risk evaluation examines the uncertainties that may affect the expected cost, benefits, schedule, technical feasibility, or overall success of the project.**

For example, a project may have an attractive expected ROI but may depend on an untested technology. Similarly, expected benefits may depend on market conditions that are uncertain. Such factors should be considered before the project is approved.

Risk evaluation mainly considers the **probability of a risk occurring and its potential impact** on the project. Important risks can then be compared with the expected benefits to determine whether the project is acceptable.

Thus, project selection should not be based only on the project with the highest expected financial return. A project with slightly lower financial benefits but significantly lower risk may be a better choice.

4.6 Selection of Appropriate Project Approach

After evaluating the project, the organization must decide **how the project should be developed**. This involves selecting suitable technologies, a software process model, and appropriate development methods.

The selection should depend on factors such as the stability of requirements, project size, technical complexity, level of risk, customer involvement, available skills, development time, and organizational environment.

4.6.2 Choosing Technologies

Technology selection involves choosing the programming languages, frameworks, databases, platforms, hardware, development tools, and other technologies required for the project.

The newest technology is not necessarily the best choice. A suitable technology should satisfy the project's functional and technical requirements while remaining practical for the organization.

- Important considerations include:
- Availability of skilled developers
- Compatibility with existing systems
- Development and maintenance cost
- Performance and scalability
- Security and reliability
- Technology maturity and vendor support
- Ease of integration and future maintenance

A technology that is technically powerful but unfamiliar to the development team may introduce unnecessary cost and risk. Therefore, technology selection should balance technical suitability, cost, skills, risk, and long-term maintainability.

4.6.2 Choice of Process Models

The **software process model** determines how major development activities such as requirements analysis, design, implementation, testing, and delivery are organized.

Different projects require different process models.

- **Waterfall Model:** Suitable when requirements are stable and well understood and major changes are not expected. Development activities are organized into defined stages.

WATERFALL MODEL

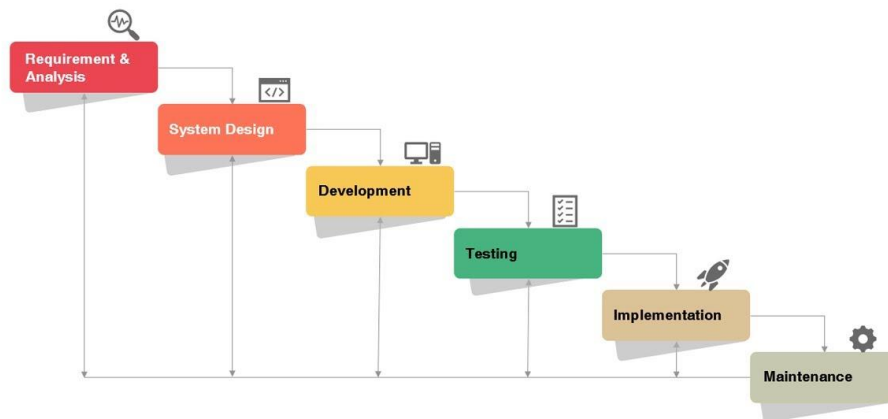


Figure ref link: https://www.youtube.com/watch?v=mY-UiRq_F-g

- **Spiral / Risk-Driven Model:** Suitable for large, complex, and high-risk projects. Development proceeds through repeated cycles with particular attention to identifying and reducing risks.

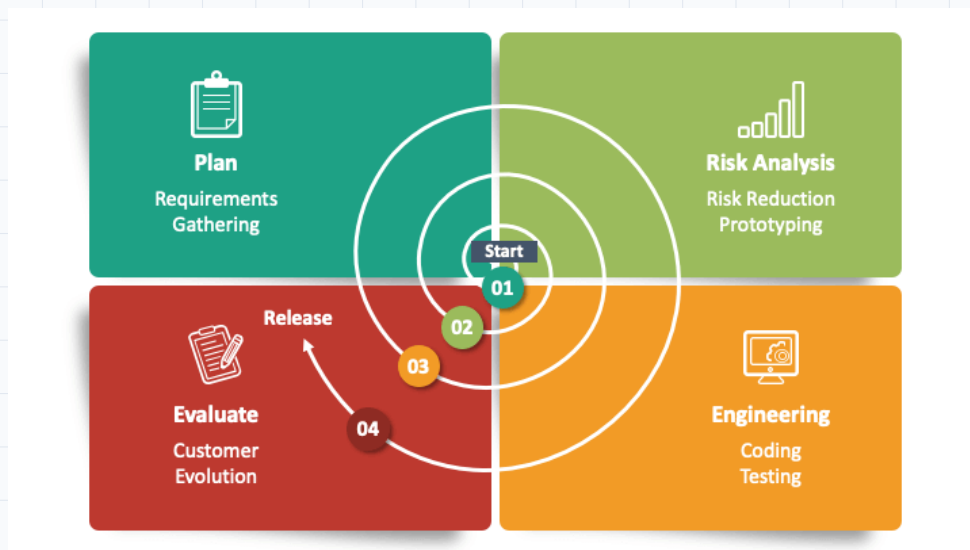


Figure Link: <https://www.sketchbubble.com/en/presentation-spiral-model-in-sdlc.html>

- **Agile Methods:** Suitable when requirements may change frequently and continuous customer feedback is important. Development is performed in smaller increments, allowing working software to be delivered and evaluated regularly.

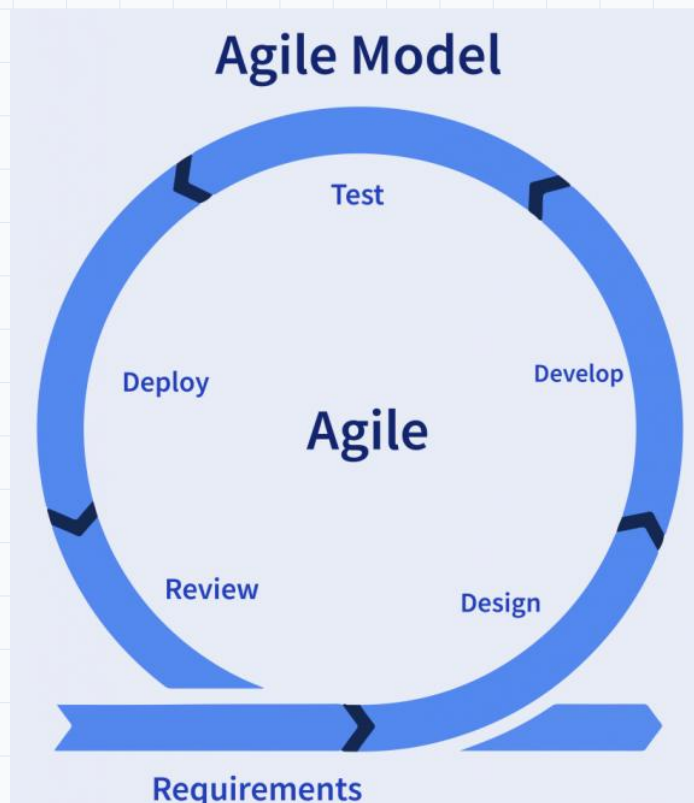


Figure Link: <https://www.linkedin.com/pulse/agile-model-darshika-srivastava/>

Therefore, the process model should be selected according to the **nature, requirements, risk, and environment of the project**, rather than using the same model for every software project.

4.6.3 Structured Methods

Structured methods provide systematic techniques for analyzing and designing software systems. They help the development team represent requirements, data, processes, and system components in an organized manner before implementation.

Common techniques include **Data Flow Diagrams (DFD)**, **Entity-Relationship Diagrams (ERD)**, and **Structure Charts**.

These methods are useful because they provide a clear representation of how data moves through the system, how data entities are related, and how software components are

organized. They can reduce ambiguity during analysis and design and provide a common understanding between users, analysts, and developers.

Overall Project Evaluation and Selection

Software project selection is a combination of **strategic, technical, financial, and risk evaluation**. No single evaluation technique can provide a complete decision by itself.

A project may have a high expected financial return but poor technical feasibility. Another project may be technically easy to implement but provide little strategic value. Therefore, management must consider the project as a whole before making the final decision.

The overall evaluation attempts to answer four important questions:

Is the project strategically valuable? → Is it technically feasible? → Is it financially worthwhile? → Are its risks acceptable?

A project that provides sufficient business value, can be implemented with available or obtainable technology and resources, provides acceptable financial returns, and has manageable risks is more suitable for selection.

Laughing Through the Chaos – Unit 4 Reality Check

Before any software project is allowed to spend real money, it must survive a polite interrogation called project evaluation. Strategic assessment asks the most dangerous question of all: "Does this actually help the business, or did someone just fall in love with a shiny new technology?" Technical assessment then asks the second most dangerous question: "Do we even have people who know how to build this, or are we about to invent a new form of suffering?"

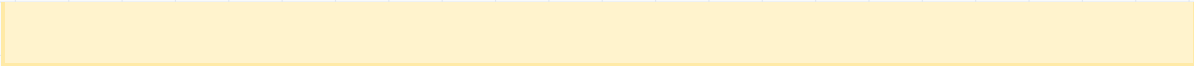
Once those two filters are passed, the real comedy begins with Cost-Benefit Analysis. In theory we subtract costs from benefits and smile if the number is positive. In

practice the costs are painfully clear (salaries, licenses, servers, late-night pizza) while the benefits are often vague poetry about "improved customer experience" and "digital transformation." Cash-flow forecasting tries to predict when money will leave the company in large amounts and when it might slowly return, which usually looks like a long, painful valley followed by a hopeful hill that may or may not appear.

Then come the financial beauty contests. Payback Period tells you how long it will take to get your money back, ignoring everything that happens after that happy day. ROI turns the same story into a percentage so management can compare projects like competing cricket scores. NPV is the sophisticated cousin that reminds everyone that money received three years from now is worth less than money in your hand today, and then discounts the future until some projects quietly disappear. A positive NPV is celebrated; a negative one is usually explained away with the phrase "but strategically important."

Risk evaluation is the final reality check. A project can look beautiful on paper and still depend on an untested technology, a single key developer, or a market that has not yet decided it wants the product. Choosing the process model is equally dramatic: Waterfall for the rare project where requirements stay still long enough to be photographed, Spiral when everyone is already nervous, and Agile when the only certainty is that nothing will stay the same for more than two weeks. Technology selection follows the same logic — the newest shiny tool is rarely the safest one, and the safest one is rarely the one the developers are excited about.

In the end, selecting a software project is less about finding the perfect investment and more about answering four uncomfortable questions without lying: Is it strategically useful? Can we actually build it? Will the money eventually come back? And can we survive the risks without needing therapy?



UNIT 5: Risk Management [5 Hrs.]

5.1 Risk Concepts & Categories

A **risk** is an uncertain event or condition that may affect the objectives of a software project if it occurs. A risk may have a **negative effect (threat)**, such as increasing cost or delaying the project, or a **positive effect (opportunity)**, such as reducing development effort or creating a new business benefit.

Risk is different from a problem. **A risk is something that may happen in the future,** whereas a problem is an issue that has already occurred. **For example, the possibility that a key developer may leave the project is a risk; if the developer has already left, it becomes an actual project problem.**

Software projects are particularly exposed to risk because requirements, technologies, people, costs, schedules, and business conditions can change during development. Risk management therefore attempts to identify important uncertainties early and prepare suitable responses before they seriously affect the project.

5.2 Categories of Risk

Software project risks can be grouped into different categories depending on what they affect.

5.2.1 Project Risks

Project risks affect the management and successful completion of the project, particularly its **schedule, cost, resources, and management activities**. Examples include:

- Staff turnover or loss of key personnel
- Unrealistic schedule or budget

- Lack of required resources
- Poor estimation
- Communication and coordination problems

For example, if a key developer leaves during an important phase, the project may require additional time to transfer knowledge or find a replacement.

5.2.2 Product / Technical Risks

Product or technical risks are associated with the software product, technology, architecture, development tools, and technical implementation. They may affect the quality, performance, reliability, security, or functionality of the final system.

Examples include:

- Use of an untested technology
- Third-party API instability
- Database or performance problems
- Difficult system integration
- Security vulnerabilities
- Architectural limitations

Technical risks can be particularly serious when the development team has little previous experience with the selected technology.

5.2.3 Business Risks

Business risks arise from changes in the organization's business environment and may affect the commercial success or continued value of the software project. Examples include:

- Changes in customer requirements

- Loss of expected funding
- Changes in market conditions
- Strong competition
- Competitors releasing similar products
- Changes in organizational priorities

For example, a project may be technically successful but still fail to provide expected business value if a competitor launches a similar product before it is released.

5. Risk Management Framework (5 Steps)

The standard risk management process involves five continuous phases:

1. Risk Identification

Risk identification is the process of discovering and documenting potential risks that may affect the project. It should be performed early and repeated throughout the project because new risks can appear as the project progresses.

Risk identification can use information from previous projects as well as the knowledge of project team members and stakeholders.

Common techniques include:

- **Brainstorming:** Team members discuss possible risks based on their knowledge and experience.
- **Checklists:** Previously identified risks are used as a reference to identify similar risks.
- **SWOT Analysis:** Strengths, Weaknesses, Opportunities, and Threats are examined to identify possible risks and opportunities.
- **Historical Information:** Risks and problems from previous projects are reviewed to identify situations that may occur again.

- **Expert Judgment:** Subject-matter experts use their professional knowledge and experience to identify risks that may not be obvious to the project team.
- **Delphi Technique:** A group of experts independently identifies and evaluates risks anonymously. Their responses are combined and reviewed through several rounds until a common understanding is reached.
- **Scenario Analysis:** Different possible future situations are considered—for example, delays in delivery, changes in requirements, or loss of key staff—to identify risks and their possible effects.
- **Cause-and-Effect Analysis:** Possible causes and their effects are examined to discover sources of risk.
- **Document Analysis:** Project documents such as contracts, schedules, budgets, and requirements are reviewed for potential risks.

The identified risks are normally recorded in a **Risk Register**, which provides a central record for monitoring and managing project risks.

2. Risk Analysis and Evaluation

After risks have been identified, they must be analyzed to determine which risks require the greatest attention. Not every identified risk has the same importance.

Two important factors are:

Probability (P): The likelihood that the risk will occur.

Impact (I): The amount of damage or effect the risk could have on the project if it occurs.

A simple measure of risk exposure is:

$$RE = P \times I$$

Where:

RE = Risk Exposure

P = Probability of occurrence

I = Impact of the risk

For example, a risk with a high probability but small impact may require less attention than a risk with a moderate probability and extremely serious impact.

Risk evaluation therefore helps the project manager **prioritize risks** and concentrate resources on the risks that can cause the greatest effect on project objectives.

3. Risk Response Planning

Once important risks have been evaluated, appropriate responses are planned. The objective is not necessarily to eliminate every risk, because some risks cannot be completely removed. Instead, the project manager attempts to **avoid, reduce, transfer, or prepare for the effects of risks**.

For negative risks or threats, the major response strategies are:

- **Risk Avoidance:** Risk avoidance involves changing the project plan, scope, technology, or approach so that the risk is eliminated or no longer applies. For example, if an untested technology creates a major technical risk, the project may use a more mature technology instead. Avoidance is useful when the potential impact of the risk is too high to accept.
- **Risk Mitigation:** Risk mitigation involves taking proactive actions to reduce either the probability of the risk occurring or its impact if it occurs. For example, automated testing, code reviews, prototypes, staff training, and backup systems can reduce the likelihood or effect of certain technical risks. Mitigation does not completely remove the risk; it makes the risk more manageable.
- **Risk Transfer:** Risk transfer involves shifting responsibility for managing a risk or its financial consequences to another party. Examples include:
 - Outsourcing specialized work
 - Using a third-party service provider

- Purchasing insurance where appropriate

The risk itself may still exist, but its consequences or ownership are transferred to another organization or party.

- **Risk Acceptance:** Risk acceptance means recognizing that a risk exists but deciding not to take specific preventive action because the cost of addressing it may be greater than its expected impact. Acceptance does not mean ignoring the risk. The project team may prepare a contingency reserve, additional time, or backup plan so that it can respond if the risk actually occurs. For example, a project manager may accept the possibility of minor schedule delays and include a suitable time buffer in the project schedule.

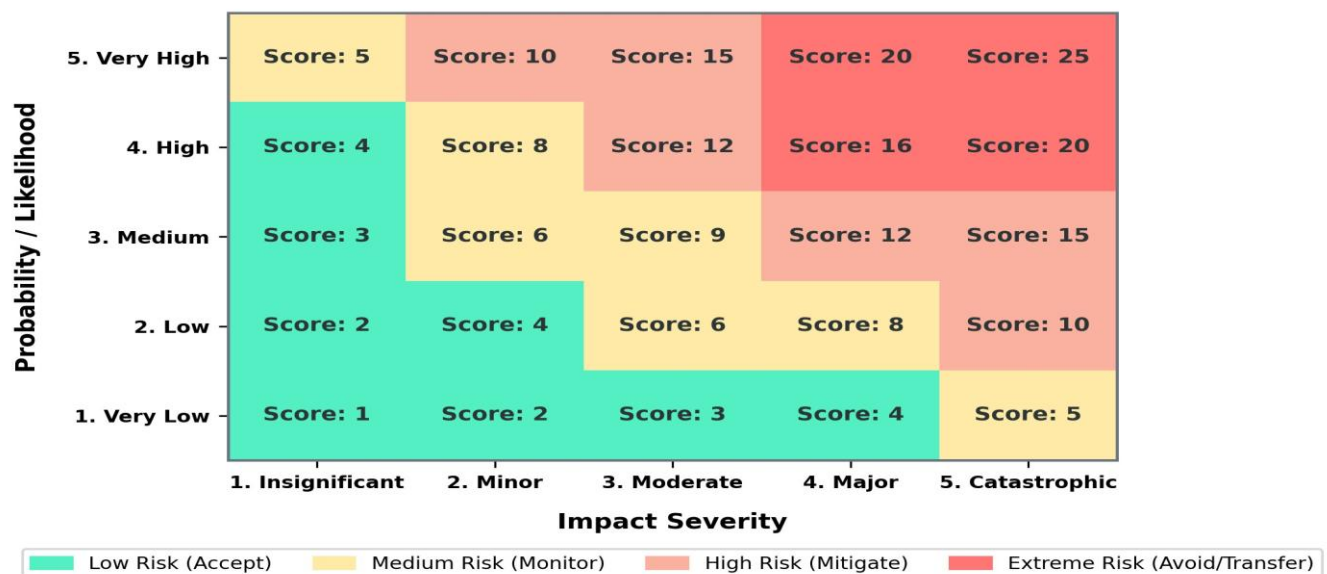
4. Risk Monitoring and Control

Risk management does not end after a response plan has been created. Risks must be continuously monitored throughout the project because their probability and impact may change over time. The project manager monitors:

- Whether identified risks are becoming more or less likely
- Whether risk impacts have changed
- Whether planned responses are working
- Whether new risks have appeared
- Whether risk triggers or warning signs have occurred

The Risk Register can be updated throughout the project to record the current status, responsible person, response strategy, and changes in risk level.

Effective monitoring allows the team to act before a risk becomes a major problem.

Figure 5.1: 5x5 Risk Assessment & Exposure Matrix

5.3 Risk Response Strategies for Threats

Strategy	Action & Mechanics	Example
Avoidance	Change project scope or plan to eliminate the risk completely.	Dropping a risky unverified feature from the release.
Mitigation	Proactive steps to reduce probability or impact severity.	Writing comprehensive automated tests to reduce bug risks.
Transferral	Shift financial impact/risk ownership to a third party.	Buying insurance or outsourcing specialized modules.
Acceptance	Acknowledge risk and set up contingency budget/schedule buffers.	Creating a 2-week schedule buffer for unexpected delays.

Laughing Through the Chaos – Unit 5 Reality Check

Risk in software projects is simply the universe's way of reminding you that nothing is ever fully under control. A risk is something that *might* happen and ruin your week; a problem is the same thing after it has already happened and is now sitting in your inbox with an angry subject line. The difference is timing, caffeine levels, and how much of your weekend remains intact.

Software risks come in three popular flavours. Project risks attack the schedule, budget, and people — the classic “key developer resigns right before the critical module” special. Technical risks attack the product itself: untested technology, unstable APIs, databases that suddenly decide performance is optional, and architectures that looked elegant on the whiteboard but collapse under real users. Business risks are the silent killers — the market changes, a competitor launches first, funding quietly disappears, or the client realises they no longer want the thing they begged you to build.

The official response is a five-step ritual that sounds professional and feels like controlled panic. First we identify risks by brainstorming, checklists, old project scars, and the occasional honest conversation. Then we analyse them with the sacred formula $\text{Risk Exposure} = \text{Probability} \times \text{Impact}$, which is just a polite way of ranking nightmares. The really dangerous ones get response plans: Avoidance (change the plan so the risk no longer exists), Mitigation (do something useful so it hurts less), Transfer (make it someone else's problem through outsourcing or insurance), or Acceptance (admit defeat early and keep a contingency buffer so the damage is survivable). Finally we monitor everything because risks have a nasty habit of evolving, multiplying, or appearing from nowhere the moment you look away.

In the end, good risk management is not about eliminating uncertainty. It is about making sure that when the inevitable chaos arrives, you already have a plan, a buffer, and enough emotional distance to say, “Yes, we saw this coming,” instead of “How is this even possible?”

UNIT 6: Software Quality Management [5 Hrs.]

6.1 Software Quality, TQM & Six Sigma

Software quality refers to the degree to which software satisfies its specified requirements, follows applicable standards, and meets the needs and expectations of its users. Quality is not limited to whether the software works correctly; it also includes characteristics such as reliability, usability, efficiency, maintainability, and portability.

Software quality should be considered throughout the software development process rather than checked only after coding is completed. Poor quality can lead to defects, rework, customer dissatisfaction, increased maintenance cost, and failure of the software to meet its intended purpose.

6.1.1 Total Quality Management (TQM)

Total Quality Management (TQM) is an organization-wide approach to improving quality continuously. It considers quality as the responsibility of everyone involved in the organization rather than only the testing or quality assurance team. The main ideas of TQM are:

- **Customer focus:** Understanding and satisfying customer requirements.
- **Continuous improvement:** Continuously improving processes and products rather than waiting for major failures.
- **Employee involvement:** Encouraging employees at different levels to participate in quality improvement.
- **Process improvement:** Improving the development process so that defects are prevented rather than repeatedly corrected.
- **Management commitment:** Management must provide leadership, resources, and support for maintaining quality.

In software development, TQM aims to build a culture where quality is considered during requirements, design, coding, testing, delivery, and maintenance.

6.1.2 Six Sigma

Six Sigma is a disciplined, data-driven approach for improving processes and reducing defects and variation. It focuses on identifying the causes of problems, measuring process performance, and making improvements based on evidence rather than assumptions. A commonly associated Six Sigma target is **3.4 defects per million opportunities (DPMO)** under the standard Six Sigma assumptions.

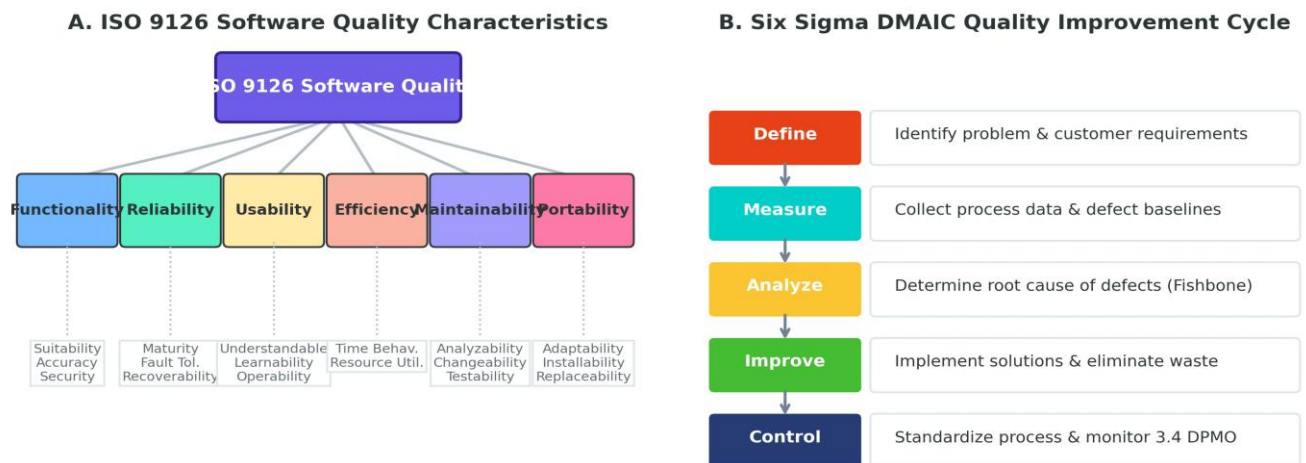
The commonly used **DMAIC** improvement cycle consists of:

- **Define:** Clearly define the problem, objectives, and customer requirements.
- **Measure:** Collect data to understand the current performance of the process.
- **Analyze:** Identify the major causes of defects or process problems.
- **Improve:** Develop and implement solutions to remove or reduce the causes.
- **Control:** Monitor the improved process to ensure that the improvement is maintained.

Thus, TQM provides a broader organization-wide quality philosophy, while **Six Sigma** provides a **structured, measurement-based approach for improving processes and reducing defects**.

6.2 ISO 9126 Software Quality Model

ISO 9126 is an international software quality model that provides a structured way of describing and evaluating software quality. It identifies **six main quality characteristics**, each representing an important aspect of software quality:

Figure 6.1: Quality Management Standards (ISO 9126 & Six Sigma DMAIC)

Quality Characteristic	Meaning
Functionality	The ability of software to provide functions that satisfy stated and implied requirements.
Reliability	The ability of software to maintain its required performance under specified conditions for a given period.
Usability	The ease with which users can understand, learn, operate, and use the software.
Efficiency	The ability of software to provide appropriate performance relative to the resources used.
Maintainability	The ease with which software can be analyzed, modified, corrected, or improved.
Portability	The ability of software to be transferred from one environment to another.

1. Functionality

Functionality refers to whether the software provides the required functions correctly and appropriately. It includes aspects such as suitability, accuracy, interoperability, security, and compliance with relevant requirements. For example, a banking application should correctly perform functions such as account management, fund transfer, and transaction processing while protecting user information.

2. Reliability

Reliability refers to the ability of software to perform consistently under specified conditions. Reliable software should operate correctly for an expected period without frequent failures. Reliability is particularly important in systems such as banking, healthcare, transportation, and other applications where software failure can have serious consequences.

3. Usability

Usability describes how easily users can understand and operate the software. A system may provide all required functions but still be considered poor quality if users find it confusing or difficult to use. Good usability involves understandable interfaces, ease of learning, ease of operation, and appropriate user guidance.

4. Efficiency

Efficiency concerns the relationship between the performance of software and the resources it consumes. Important resources may include processor time, memory, storage, network capacity, and other system resources. For example, two applications may provide

the same functionality, but the application that responds faster while using fewer system resources may have better efficiency.

5. Maintainability

Maintainability refers to how easily software can be understood, corrected, modified, or enhanced after it has been developed. Software is often changed because of new requirements, defects, technology changes, or improvements. Well-structured and well-documented software is generally easier and less costly to maintain.

6. Portability:

Portability refers to the ability of software to be transferred from one operating environment to another with reasonable effort. For example, software may need to operate on different operating systems, hardware platforms, or environments. Software with good portability can be adapted to such environments more easily. The ISO 9126 model therefore provides a common framework for discussing software quality instead of treating quality simply as the absence of errors.

6.3 Place of Software Quality in Software Planning

Software quality should be **planned from the beginning of a project** rather than treated as an activity performed only after development is complete. If quality is considered only during final testing, defects may already be deeply embedded in requirements, design, architecture, or code and may therefore be more expensive to correct.

Quality planning determines **what level of quality is expected, how it will be achieved, and how it will be measured** throughout the project. Important quality activities include:

- **Software Quality Assurance (SQA):** Establishing processes, standards, reviews, and procedures to ensure that development activities follow the required quality practices.
- **Process Audits:** Checking whether the defined development processes, standards, and procedures are actually being followed. Audits can identify gaps and areas where the process needs improvement.
- **Peer Reviews and Inspections:** Having developers or other team members examine requirements, designs, and code to identify defects early.
- **Testing:** Performing different levels of testing, such as unit, integration, system, regression, and acceptance testing, to verify that the software satisfies its requirements.
- **Automated and Continuous Testing:** Automatically running tests whenever software changes are integrated. This helps detect defects quickly and supports practices such as **Continuous Integration (CI)**.
- **Quality Metrics:** Measuring factors such as defects, rework, reliability, test results, and progress to understand the quality of the product and process.
- **Requirements Quality Management:** Reviewing requirements to ensure they are clear, complete, consistent, realistic, and testable. Poor requirements can lead to defects later in design and development.
- **Configuration and Change Management:** Controlling changes to source code, requirements, documents, and other project artifacts. Proper change management prevents unauthorized or conflicting changes from affecting software quality.
- **Security and Performance Testing:** Evaluating whether the software is secure, responsive, reliable, and able to handle the expected workload. This is especially important for systems that handle sensitive information or large numbers of users.
- **Customer and User Feedback:** Collecting feedback from users and stakeholders during and after development. Feedback can reveal usability problems, missing requirements, and other quality issues that technical testing may not identify.

Quality planning is closely connected with other project activities. For example, additional testing may affect the schedule and cost, while poor quality may increase rework and maintenance effort. Therefore, the project manager must consider **quality, time, cost, scope, and resources together.**

Walker Royce's 7 Core Metrics:

Walker Royce identified a set of core project metrics that can help managers monitor software project performance and identify potential problems.

The seven commonly referenced metrics are:

- **Work and Progress:** Measures the amount of work completed and the progress being made toward project objectives.
- **Budgeted Cost:** Tracks planned and actual project expenditure.
- **Staffing Dynamics:** Monitors how the number and composition of project staff change over time.
- **Change Traffic:** Measures the amount and rate of change in requirements or other project artifacts.
- **Breakage:** Tracks defects, failures, or other forms of damage introduced during development.
- **Rework:** Measures effort spent correcting or repeating previously completed work.
- **MTBF / Maturity:** Measures reliability and the maturity of the software, commonly using **Mean Time Between Failures (MTBF)** as an indicator.

These metrics help the project manager identify whether the project is progressing normally or whether increasing defects, rework, changes, costs, or staffing problems indicate the need for corrective action.

Therefore, software quality management is not simply about **finding defects**. Its broader objective is to **prevent defects, improve development processes, measure quality, and**

ensure that the final software satisfies both technical requirements and user expectations.

Laughing Through the Chaos - Unit 6 Reality Check

Software quality is the difference between a system that works and a system that people don't actively curse while using. It is not just "does it run without crashing," but whether it is reliable enough to trust, usable enough that users don't need a training course and a therapist, efficient enough not to melt the servers, maintainable enough that future developers don't cry, and portable enough that it doesn't fall apart the moment someone changes the operating system.

Total Quality Management is the organisational version of "quality is everyone's problem." Instead of dumping everything on the testing team at the end, TQM insists that analysts, designers, developers, managers, and even the person who orders the coffee all share responsibility. Six Sigma takes the same idea and adds spreadsheets, statistics, and the ambitious target of only 3.4 defects per million opportunities. It follows the DMAIC ritual: Define the mess, Measure how bad it is, Analyse why it happens, Improve it, and Control it so the mess doesn't quietly return the moment everyone looks away.

ISO 9126 gives us a neat checklist of what "good" actually means: Functionality (it does what it claims), Reliability (it keeps doing it), Usability (humans can use it without rage), Efficiency (it doesn't waste resources), Maintainability (someone can still fix it in two years), and Portability (it survives moving to a new environment). The painful truth is that quality has to be planned from day one. Waiting until the final testing phase is like checking whether the house is waterproof after the roof is already finished and the monsoon has started.

In practice this means reviews, inspections, continuous testing, clear requirements, controlled changes, and metrics that actually tell you something useful — work progress, cost, staffing chaos, change traffic, breakage, rework, and how long the system survives between failures. Ignore these and you end up with software that technically "works"

but still manages to disappoint everyone who touches it. Good quality management is simply the art of preventing future versions of yourself from hating past versions of yourself.

UNIT 7: Software Configuration Management

7.1 Concept, Need and Elements of SCM

Software Configuration Management (SCM) is the process of identifying, organizing, controlling, and tracking changes made to software and its related artifacts throughout the software development lifecycle. These artifacts may include source code, requirements, design documents, test cases, configuration files, and user documentation.

During software development, many changes occur. Different developers may work on the same files, requirements may change, defects may need to be corrected, and new versions of the software may be created. Without proper configuration management, it becomes difficult to determine which version is correct, what was changed, who made the change, and why the change was made.

The main purpose of SCM is therefore to maintain the integrity, consistency, traceability, and controlled evolution of software products.

Need for Software Configuration Management

SCM is needed because software projects involve continuous changes and multiple people working on shared artifacts. Proper SCM helps to:

- Control changes and prevent unauthorized modifications.
- Maintain different versions of software and documents.
- Identify the exact version used for development, testing, or release.
- Prevent developers from accidentally overwriting each other's work.
- Maintain a history of changes for tracking and troubleshooting.
- Provide a stable and agreed basis for further development.
- Support reliable software releases and maintenance.

For example, if a defect is discovered in a released version, SCM allows the team to identify the exact version of the source code, determine what changes were introduced, and make a controlled correction.

DEFINITION: 4 Elements of SCM & Baselines

1. **Configuration Identification:** Configuration identification determines **what items need to be controlled and how they will be uniquely identified**. These items are called **Configuration Items (CIs)** and may include source code, requirements, design documents, test cases, and executable files. Each configuration item is given an appropriate name, identifier, or version so that it can be distinguished from other items.
2. **Configuration Change Control:** Configuration change control ensures that changes to controlled software items are **proposed, evaluated, approved, implemented, and recorded in a controlled manner**. A change should not simply be made whenever someone requests it. Its effect on cost, schedule, quality, functionality, and other components should first be considered. In formal environments, changes may be reviewed and approved by a **Configuration Control Board (CCB)** or another authorized group. The basic idea is: **Change Request → Evaluation → Approval / Rejection → Implementation → Verification → Recording**. This prevents uncontrolled changes from affecting the stability of the project.
3. **Configuration Status Accounting:** Configuration status accounting involves **recording and reporting the current status and history of configuration items**. It can provide information such as: Which version of an item currently exists, What changes have been made, Who made the changes, When a change was made, Which changes have been approved, Which version has been released. This information

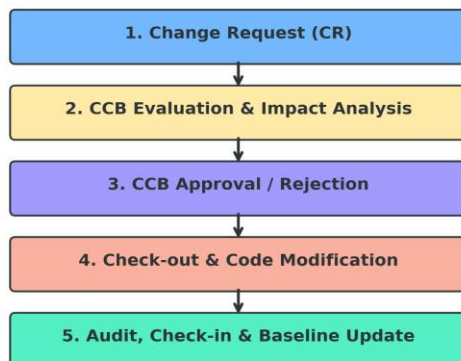
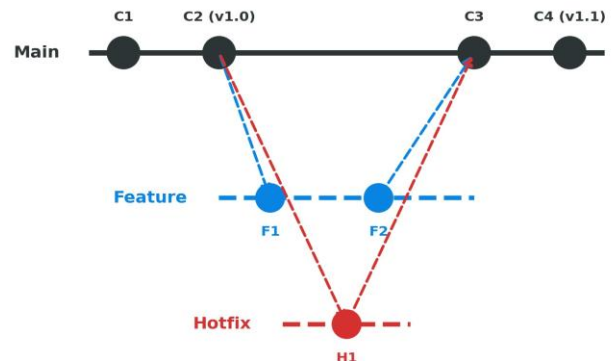
provides traceability and helps project managers and developers understand the current state of the software.

4. **Configuration Audit:** A configuration audit checks whether the software configuration is complete, correct, and consistent with the approved requirements and configuration records. Two commonly discussed types are: **Functional Configuration Audit (FCA):** Verifies that the software performs the functions specified in its requirements. **Physical Configuration Audit (PCA):** Verifies that the actual software product and associated documentation correspond to the approved configuration. Audits provide additional confidence that the product being delivered is actually the product that was approved and tested.

Baseline: A baseline is a formally reviewed and approved version of a software configuration item or a group of configuration items that serves as a **stable reference point for further development**. Once a baseline is established, changes to it are normally controlled through the change control process.

For example, after requirements have been reviewed and approved, the approved requirements document may become a **requirements baseline**. If a customer later requests a major change, the change should be evaluated and approved rather than simply modifying the baseline without control.

Baselines are useful because they provide a known reference against which future changes can be compared.

Figure 7.1: SCM Process & Modern Version Control Branching Model**A. SCM Change Control Process Flow****B. Distributed SCM Branching Model (Git)**

SCM Change Control Process

Software Configuration Management (SCM) provides a systematic way to control and manage changes to software configuration items throughout the software development lifecycle. Changes may be required because of new customer requirements, defect corrections, security improvements, performance issues, or changes in business needs. The change control process ensures that changes are not made randomly or without considering their effects on the project. A typical SCM change control process consists of the following steps.

- Change Request (CR)

The process begins when a stakeholder identifies a need for a change and submits a **Change Request (CR)**. The request may come from a customer, developer, tester, project manager, or other authorized stakeholder. The change request describes what needs to be changed, why the change is required, and which software components or configuration items may be affected. It may also include the priority, expected benefits, and supporting information. The request is formally recorded so that it can

be tracked from submission to final implementation. For example, if a customer requests a new password-reset feature, the requirement is submitted as a formal change request rather than being directly added to the software.

- **CCB Evaluation and Impact Analysis**

After the change request is submitted, it is reviewed by the **Configuration Control Board (CCB)** or another authorized group. The CCB evaluates whether the requested change is necessary, feasible, and consistent with the project objectives. An important part of this stage is **impact analysis**, where the team determines how the proposed change could affect the existing system. The impact may involve cost, schedule, functionality, performance, security, quality, documentation, testing, and other configuration items. The team also considers dependencies and possible risks associated with implementing the change. For example, adding a password-reset feature may require changes to the database, user interface, authentication system, email service, security controls, and test cases.

- **CCB Approval or Rejection**

After evaluating the change and its possible impacts, the CCB makes a formal decision. The change request may be **approved, rejected, or deferred** depending on its importance, feasibility, cost, risk, and effect on the project. If the change is approved, the responsible team is authorized to implement it. If it is rejected, the reason should be documented so that there is a clear record of the decision. A change may also be deferred if it is useful but cannot be implemented at the current stage of the project. This step prevents unauthorized changes from being introduced into the software.

- **Check-out and Code Modification**

Once a change has been approved, the responsible developer or development team obtains the relevant configuration item from the configuration management system through a **check-out** process. Check-out identifies the version being modified and helps prevent uncontrolled or conflicting modifications. The developer then makes the approved changes to the source code, configuration files, documentation, or other affected configuration items. The developer should follow the organization's coding standards and development procedures while implementing the change. Appropriate developer-level testing should also be performed to make sure that the modification behaves as expected.

- **Review, Testing, and Audit/Verification**

After the modification is completed, the changed software is reviewed and tested to determine whether the approved change has been implemented correctly. **Code reviews** can be performed to identify programming errors or violations of coding standards. Testing may include unit testing, integration testing, system testing, and regression testing. Regression testing is particularly important because a new change should not unintentionally break existing functionality. A configuration audit or verification may also be performed to confirm that the implemented change matches the approved change request and that all required configuration items and documentation have been updated. If problems are identified, the software may need to be corrected and tested again before it can proceed.

- **Check-in and Baseline Update**

After the modified configuration item successfully passes the required reviews and tests, it is **checked back into the configuration management system**. The system records important information about the new version, such as who made the change,

when it was made, what was changed, and which change request authorized the modification. If the change has been formally accepted, the appropriate **baseline may be updated** to include the approved version. The baseline then represents a new stable and controlled reference point for further development. This ensures that everyone working on the project is using the correct and officially approved version of the software.

Configuration Audit and Status Reporting

Configuration audits verify that the actual software and its documentation are consistent with the approved configuration. **Status reporting** provides information about the current state of configuration items and changes. Reports may show the current versions, approved changes, pending changes, released configurations, and other configuration information.

Together, auditing and status reporting help management maintain **visibility, traceability, and control** over the software product.

7.2 Version Control Tools Case Study: Git vs. CVS vs. SVN

Version control tools are software systems used to manage changes to source code, documents, and other project files. They maintain a history of modifications, allow developers to work on different versions of a project, support collaboration, and make it possible to compare or restore earlier versions. **CVS, SVN, and Git** are important version control systems, but they differ mainly in their architecture, branching approach, offline capabilities, and method of storing changes.

Feature / Aspect	CVS (Concurrent Versions)	SVN (Subversion)	Git (Distributed SCM)
Architecture	Centralized server	Centralized server	Distributed / Peer-to-Peer

Repository Copy	Central server only	Central server only	Full repository on every local client
Branching Speed	Very slow & heavy	Slow (directory copies)	Lightweight & near-instant
Offline Operation	No (requires network)	No (requires network)	Full offline commits, diffs & logs
Atomic Commits	No (file-by-file)	Yes (revision number)	Yes (SHA-1 changeset DAG)

- **CVS**

CVS (Concurrent Versions System) is an older centralized version control system. It uses a central repository where the project's version history is maintained. Developers normally need to communicate with the central server for many version-control operations. CVS supports collaborative development and maintains file revision history, but its branching and merging capabilities are relatively limited compared with newer systems.

- **SVN**

SVN (Subversion) is also a centralized version control system, but it provides several improvements over CVS. It maintains the main repository on a central server and supports atomic commits, versioned directories, branching, and tagging. SVN is easier and more powerful to manage than CVS, but developers generally depend on access to the central repository for many operations.

- **Git**

Git is a distributed version control system. Instead of keeping the complete project history only on a central server, each developer normally has a local repository containing the project's history. This allows many operations, such as creating

commits, viewing history, comparing changes, and creating branches, to be performed locally without a network connection. Git also provides fast branching and merging, making it widely used for modern software development.

Laughing Through the Chaos - Unit 7 Reality Check

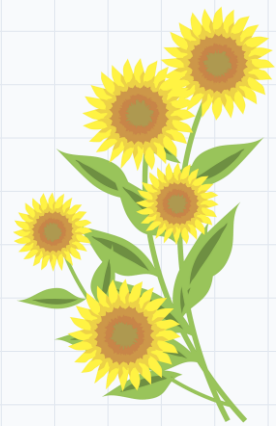
Software Configuration Management is the adult supervision that stops a software project from turning into a free-for-all where five developers edit the same file, three different "final" versions exist, and nobody can remember which one actually worked last Tuesday. Without it, source code, requirements, designs, and test cases drift around like unsupervised children at a wedding.

SCM has four main jobs. Configuration Identification decides what counts as important enough to control and gives every item a proper name and version so people stop calling everything "the latest one." Change Control is the formal process that forces a change request to go through evaluation, impact analysis, and approval before anyone is allowed to touch the code — usually via a Configuration Control Board that exists mainly to stop spontaneous "quick fixes" from becoming permanent disasters. Status Accounting keeps the boring but essential record of who changed what, when, and why. Configuration Audits then check whether the thing you are about to release actually matches what was approved and tested.

Baselines are the official "this is the stable version we all agree on" checkpoints. Once something is baselined, changing it requires permission, paperwork, and a good reason. The full change control dance goes: someone raises a Change Request → the CCB stares at the impact → it gets approved or rejected → the developer checks out the code → makes the change → reviews and tests it → checks it back in → and only then does the baseline move forward. Skip any of these steps and you eventually discover that the

production system is running a version that only existed on one person's laptop for forty minutes.

Version control tools are the practical weapons in this war. CVS was the old centralized system that worked until it didn't. SVN improved on it but still forced everyone to talk to a central server for almost everything. Git arrived, gave every developer a full copy of the history, made branching almost free, and allowed offline work, which is why most of the industry quietly migrated and never looked back. In short, good SCM is the difference between "we know exactly what we released" and "we think this might be the version that worked once."

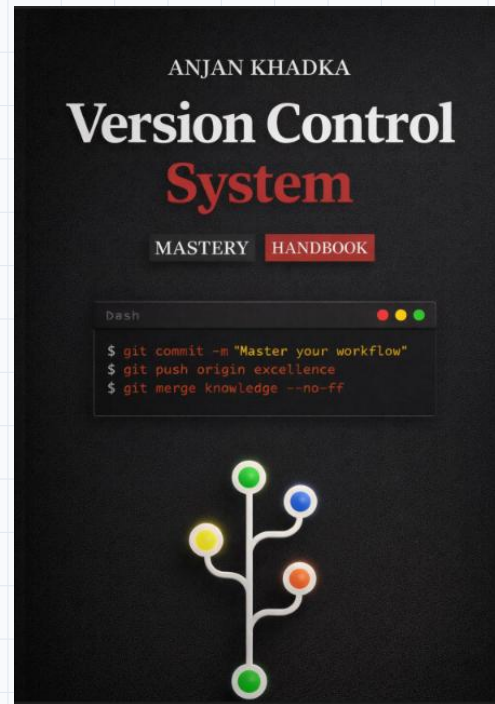


Git Learning Reference:

For students who want to learn Git in greater depth, refer to the **Git Handbook**, which provides a practical guide from basic Git concepts to advanced day-to-day operations.

The handbook covers:

- Getting started with Git and Git fundamentals
- git init, status, add, commit, and log
- git reset and git revert
- Branches and branching concepts
- git merge and resolving merge conflicts
- git cherry-pick
- Effective commit messages, empty commits, and amending commits
- Remote repositories, git clone, fetch, and pull
- git stash and .gitignore
- Types of merges and Git merge strategies



Want to Learn Git? → [Git Handbook - https://github.com/Anjan012/Git-Version-Control-System-note-/blob/main/VCS_Mastery_Book.pdf]